Preface

Dear friends,

A warm welcome to this course “Data Structures and Applications”. I am N S Kumar. I am the anchor for this course. We have a very good team delivering this course. We will together assist you in learning this subject well. We will provide you with slides, lecture notes, MCQs, short and long questions and answers. Most of these shall be shared with you on a weekly basis.

We consider this course as one of the most important courses in Computer Science curricula. We feel that mastering data structures and algorithms will not only help you in learning other subjects, but also in getting placed in a good company. Most companies as on today test your ability to think in terms of data structures and algorithms.

You may be wondering whether it is required to learn Data Structures and Algorithms in this era of Generative AI. Most of you would have used these tools in your programming courses.

As on today, these tools are generating programs. But the companies are not yet ready to put them into production before they are checked by human eyes. There is a shortage of reviewers in the market. You should be able to read programs, debug them, make them effective and efficient. You will enhance your reading skills only if you also know how to write programs.

So, we strongly recommend that you avoid using these tools while developing programs in this course. We will use the tools when appropriate. We will not only encourage you to write programs, but also to read programs.

You may contact me or any of us if you have any queries.

All the best. Enjoying learning.

On behalf of our team,

N S Kumar

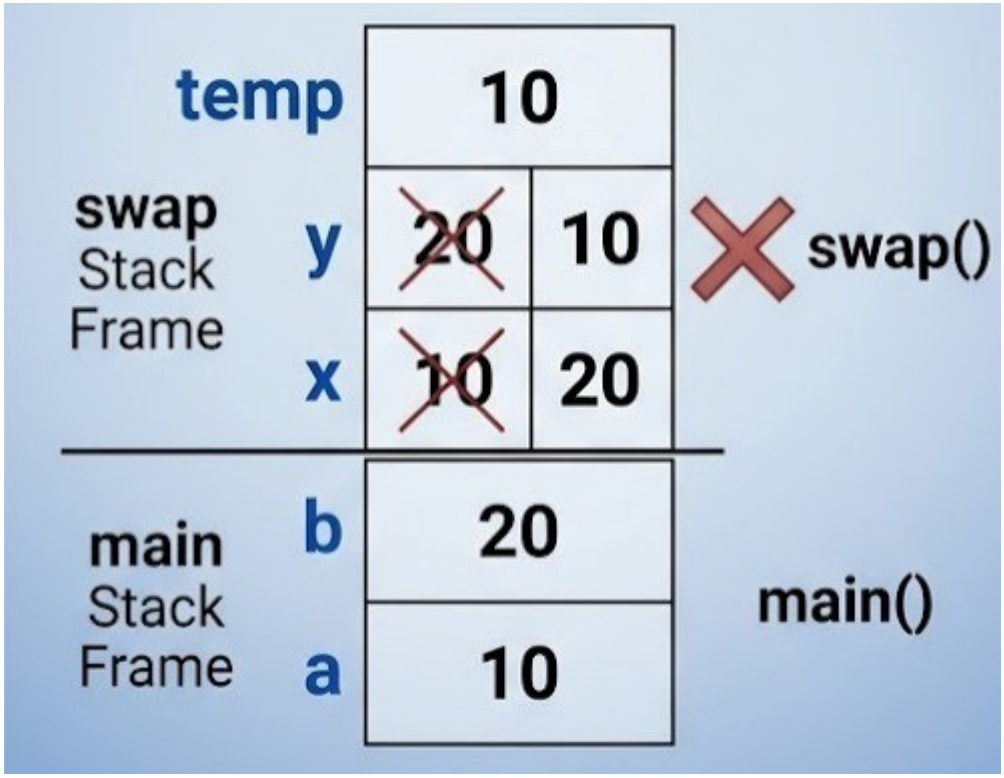
nskumar@pes.edu

Function

Let us continue our discussion of functions with the following example from 1_swap.c.

```
// Does not swap the arguments!!
void swap(int x, int y)
{
        int temp = x; x = y; y = temp;
}

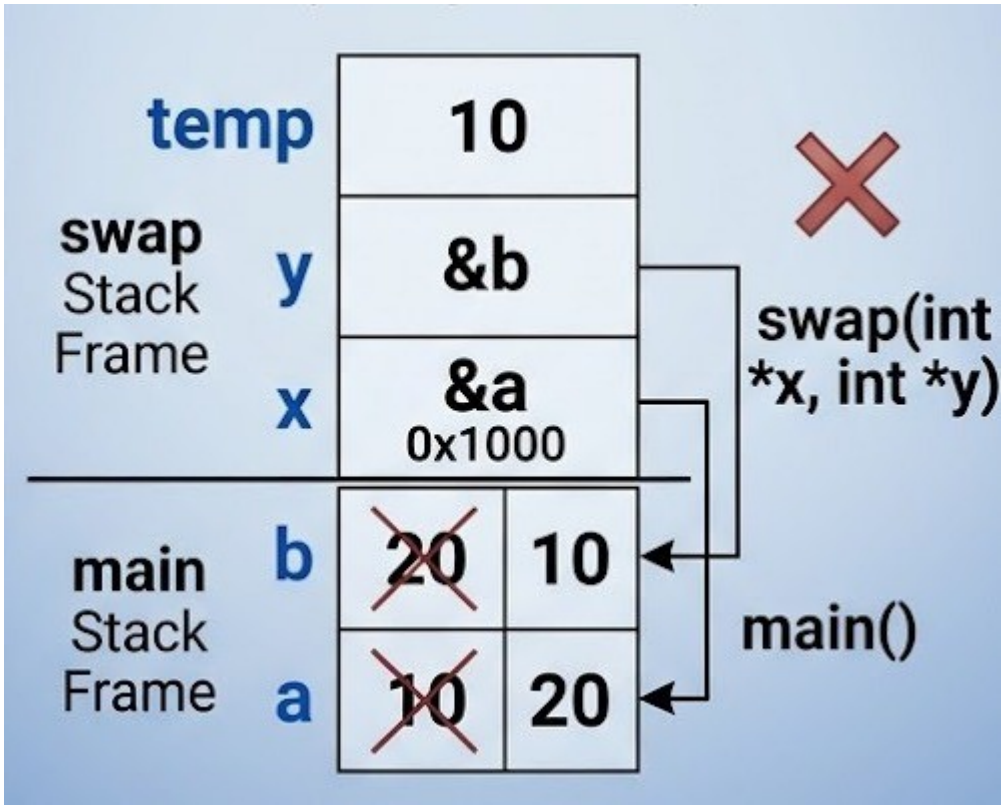
int main()
{
        int a = 10; int b = 20;
        swap(a, b);
        printf("a : %d b : %d\n", a, b);
}
```



We know that the parameter passing in 'C' is by value. So, the copies of a and b get swapped; but not a and b.

To change a variable of the caller, the callee should receive pointer to the variable.

This works!




```
void swap(int *x, int *y)
{
	int temp = *x; *x = *y; *y = temp;
}

int main()
{
	int a = 10; int b = 20;
	swap(&a, &b);
	printf("a : %d b : %d\n", a, b);
}
```

Recursion:

A function calling itself directly or otherwise is said to be recursive. We may use if and recursion instead of a looping structure. In recursion, we try to express the solution to a problem in terms of the problem itself, but of a smaller size. We also require solution for one or more simple cases – called escape hatch or base case.

Let us examine how to trace a recursive function.

In this example, there are two places where recursive calls are made. We should know where these recursive calls will return. We can mark the position of the call with the number of the call – so that we will know where to resume the following of the code.

This function finds the number of 1s in the binary representation of the number.

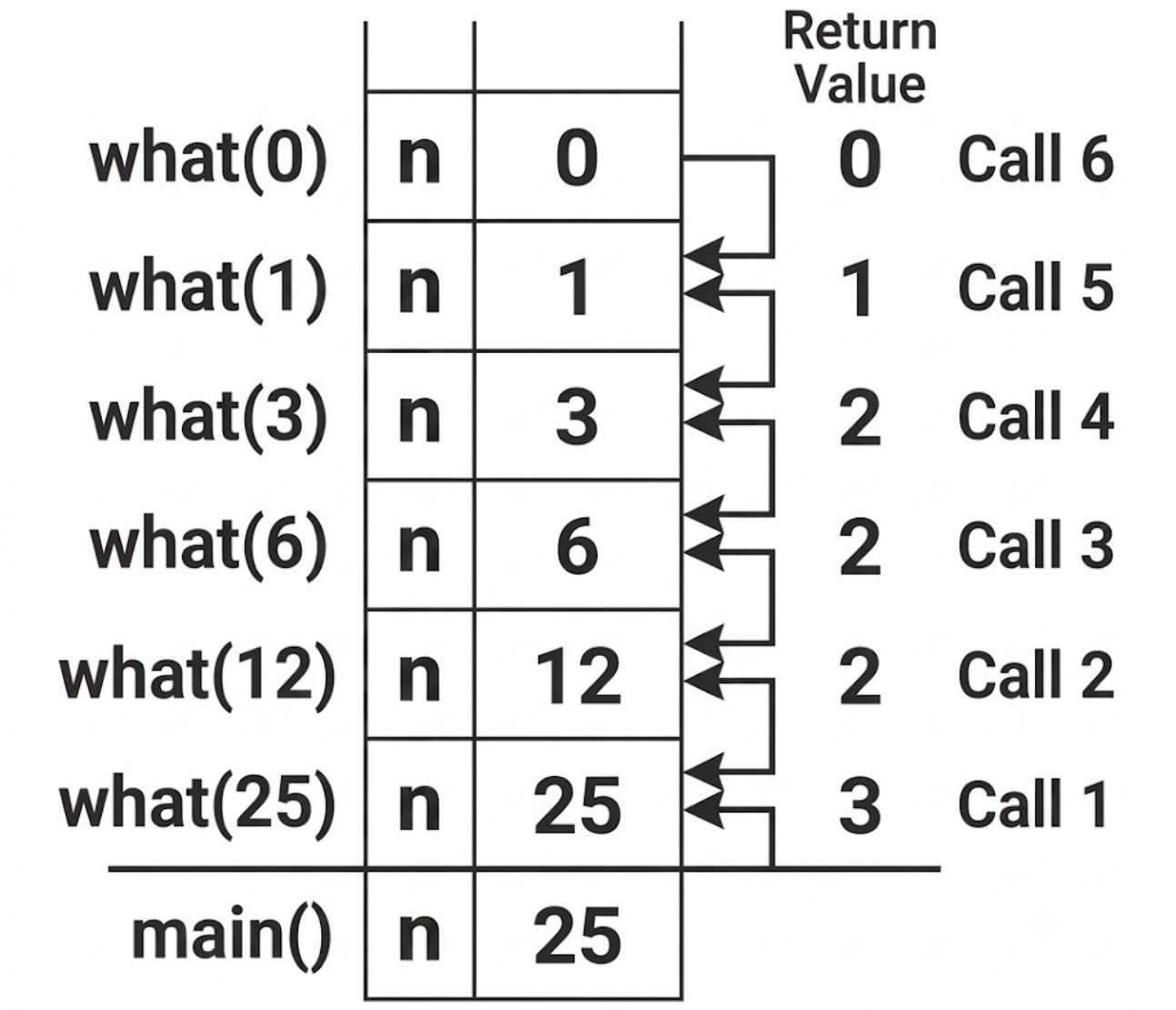
If the number is odd, we count 1 and remove the unit bit and call the function on the changed number.

If the number is even, we remove the unit bit then we call the function on the changed number.

If the number is 0, we return 0.

```
int what(int n)
{
        if(n == 0)
        {
                return 0;
        }
        else if(n % 2)
        {
                return 1 + what(n / 2);
                //      2 => 2
                //              5 => 1
                //              6 => 0
        }
        else
```

```
    {
        return what(n / 2);
        //              3 => 2
        //    4 => 2
    }
}
```



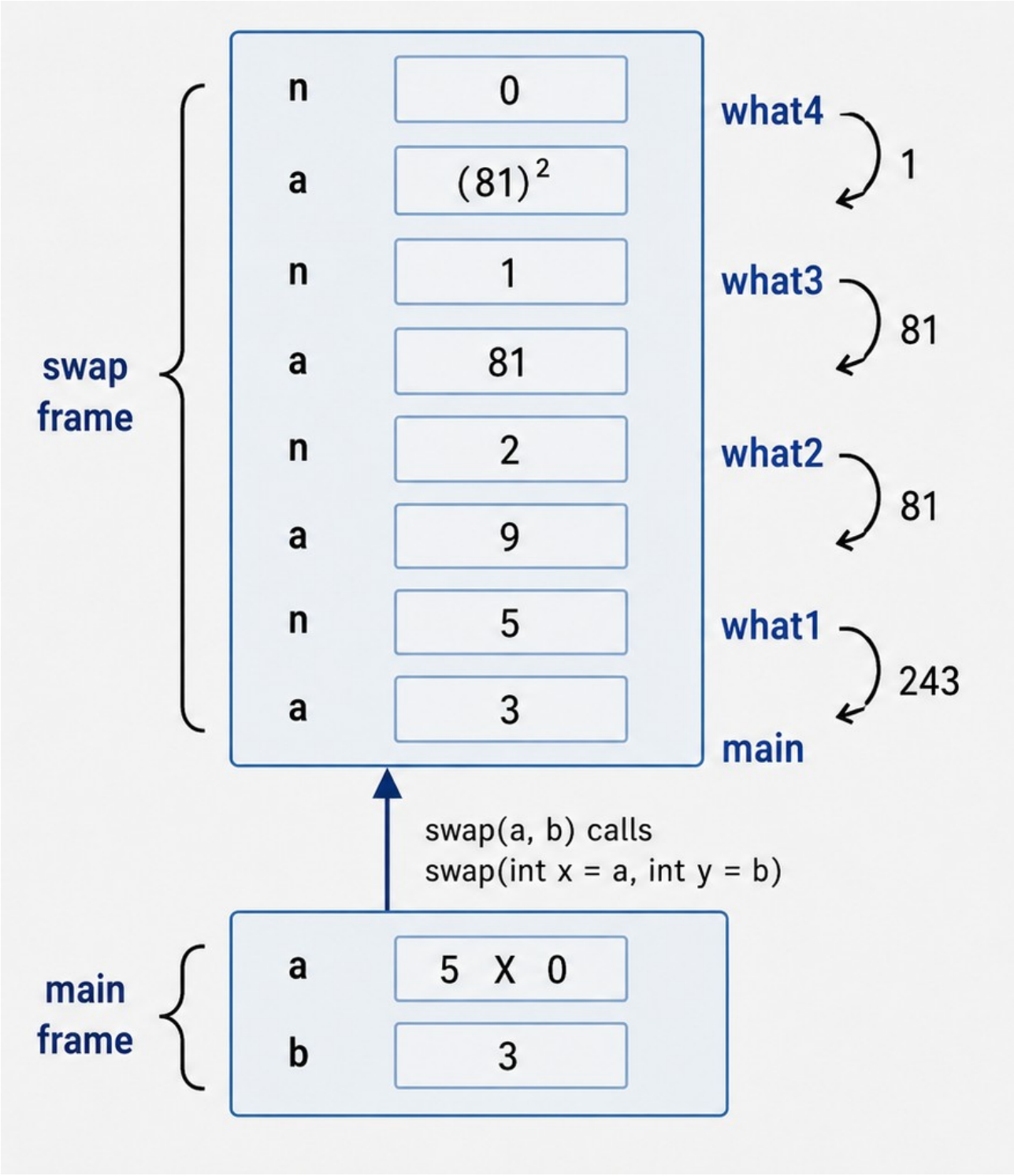

```
int main()
{
    int n  = 25;
    printf("val : %d\n", what(n));
    //                                      1 => 3
}
```

Recursion: Another example:

This is the file 3_what.c.

What does this recursive function do? You may want to pause and think about this function.

```
#include <stdio.h>
int what(int a, int n)
{
	if(n == 0)
		return 1;
	else if(n % 2)
		return a * what(a * a, n / 2);
	else
		return what(a * a, n / 2);
}
int main()
{
	int a = 3; b = 5;
	printf("%d\n", what(a, b));
}
```



It finds a to the power of n. The definition is as follows.

If n is 0, the result is 1.

if n is even, square a and then exponentiate the result to n / 2.

If n is odd, square a and then exponentiate the result to n / 2 and then multiply this with a.

Parameter passing and pointers:

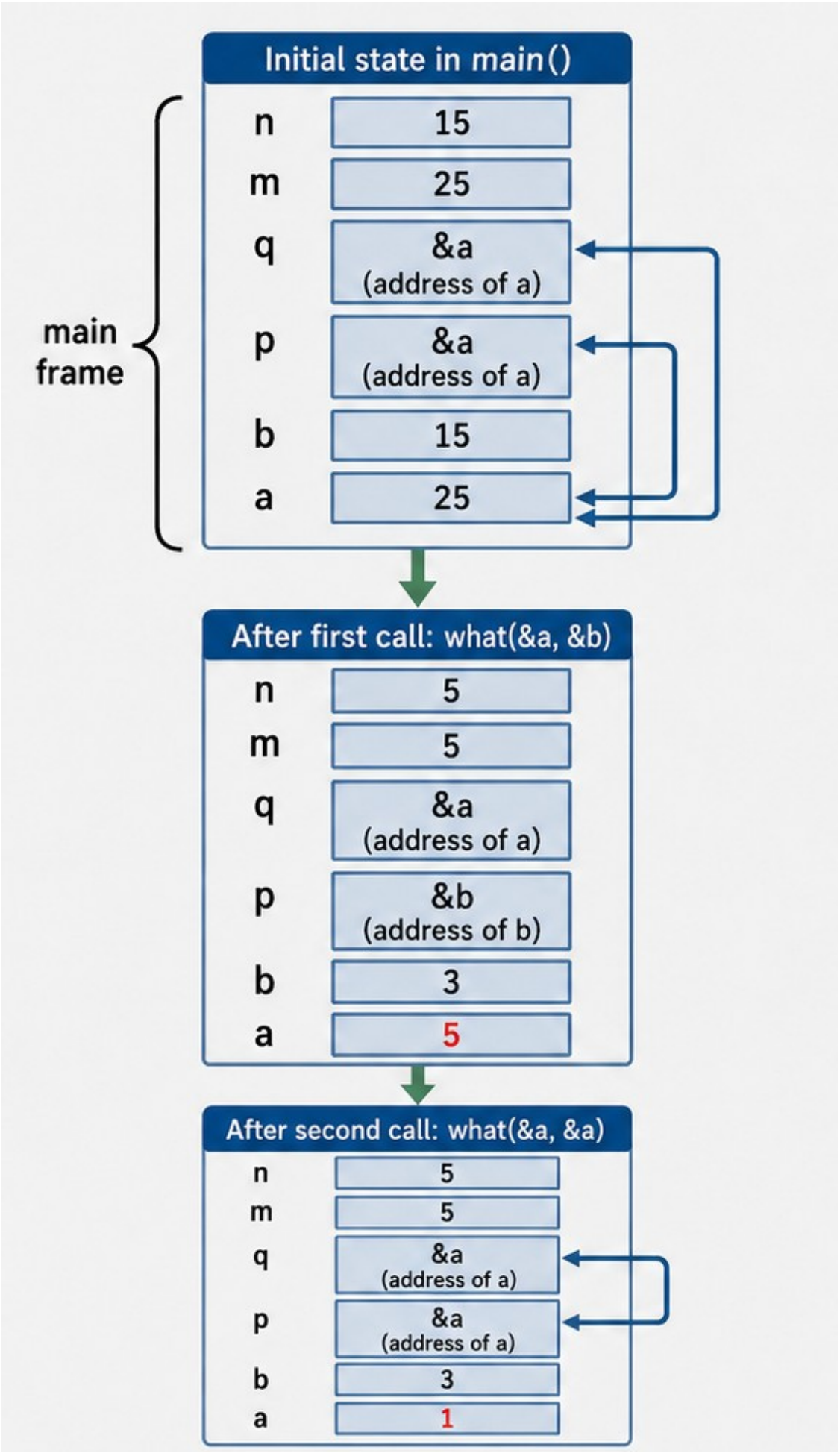
This is an interesting example of understanding parameter passing.

The loop of this function finds the greatest common divisor of m and n. p and q point to the corresponding arguments. m and n are copies of *p and *q and have nothing to do with the arguments.

*p /= m; *q /= n;

This statement causes division of the arguments by the greatest common divisor.

```
void what(int *p, int *q)
{
	int m = *p; int n = *q;
	while(m != n)
	{
		if(m > n)
		{
			m -= n;
		}
		else
		{
			n -= m;
		}
	}
	*p /= m; *q /= n;
}
int main()
{
	int a = 25; int b = 15;
	what(&a, &b);
	printf("%d %d\n", a, b);
	what(&a, &a);
	printf("%d\n", a);
}
```

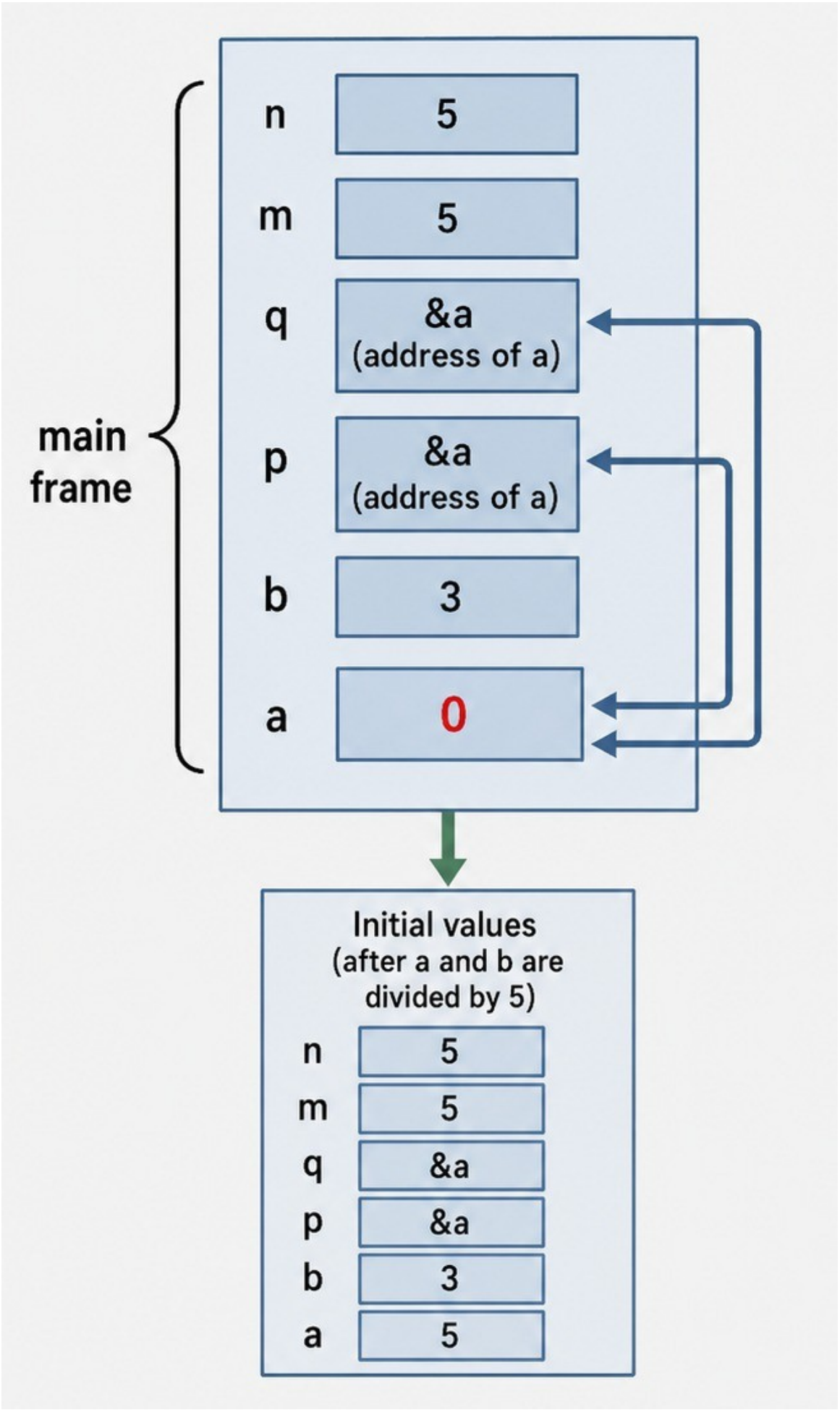

Initial state in main()
main frame
n
15
m
25
q
&a
(address of a)
p
&a
(address of a)
b
15
a
25
After first call: what(&a, &b)
n
5
m
5
q
&a
(address of a)
p
&b
(address of b)
b
3
a
5
After second call: what(&a, &a)
n
5
m
5
q
&a
(address of a)
p
&a
(address of a)
b
3
a
1

```
int a = 25; int b = 15;
what(&a, &b);
printf("%d %d\n", a, b);
```

So, a and b get divided by 5. So a becomes 5 and b becomes 3.

The next part of the code is interesting. What do you expect the variable a to become. - 1 ? Be in for a surprise!

Both p and q point to the same variable. This is called aliasing. m and n become 5. The loop is not entered. We update *p that is same as variable a by dividing by m. So a becomes 1. We update *q that is also a by dividing by n. So, a becomes 0!!


main frame
n
5
m
5
q
&a
(address of a)
p
&a
(address of a)
b
3
a
0
Initial values
(after a and b are divided by 5)
n
5
m
5
q
&a
p
&a
b
3
a
5

Separating the interface from implementation:

Our requirement let us say is to check whether a given number is a palindrome.

We can write the function ourselves. But we could use a function in the library or written by somebody as long as it satisfies the requirement.

Let us know that there is a function with the following signature.

```
int is_palindrome(int n);
```

Then we can write the code like this.

```
	int n;
	scanf("%d", &n);
	// check whether it is a palindrome
	if(is_palindrome(n))
	{
		printf("%d is a palindrome\n", n);
	}
	else
	{
		printf("%d is not a palindrome\n", n);
	}
```

The writer of the function does not know how we use. We do not know how the developer of the function has written the function. The developer may write the logic together in the same function or call some other function to reverse the number.

```
int rev(int n)
{
	int r = 0;
	while(n)
	{
		r = r * 10 + n % 10;
		n /= 10;
	}
```

```
        return r;
}
int is_palindrome(int n)
{
#if 0
        int rev = 0; int m = n;
        while(n)
        {
                rev = rev * 10 + n % 10;
                n /= 10;
        }
        return m == rev;
#endif
        return n == rev(n);
}
```

So, we separate the client code and server code. The common code is put in the header file. It contains declaration of functions. The header file acts like the interface between the client and server.

Multiple files and make file:

The next example has 3 files.

6_server.c – provides implementation of squaring and cubing

6_client.c – uses these functions squaring and cubing

6_server.h : interface for these functions.

We compile both the .c files then link them to create the loadable image.

Each time we change any of the files, we should remember to recreate the files by using appropriate commands.

Can we transfer these two ideas to a tool?

- dependency of files
- command(s) to be executed when the dependent files change.

This tool called the make tool takes care of these.

This is the file : mymakefile.mk

The first line indicates the file created last in the sequence – the loadable image(executable). This is called a target. Then this line indicates which all file it depends on. They are called sources.
The next lines(s) start with a tab and any command to be executed when the sources are modified.
This sequence of dependencies and commands repeat.
The command to be executed is : make -f mymakefile.mk.

```
 a.out : 6_client.o 6_server.o
        gcc 6_client.o 6_server.o
6_client.o : 6_client.c 6_server.h
        gcc -c 6_client.c
6_server.o : 6_server.c 6_server.h
        gcc -c 6_server.c
```

Experiment changing the files and observe that only required files are re-created.

Function

Mathematically, a function is a subset of relation. Relation is a subset of cartesian product of sets. A function associates a sequence of values – called **arguments**, with **a value** – called the **result** of the function application or the call. In Mathematics, the arguments will not change when the function is applied. Not only that, but each time the function is applied to the same sequence of arguments, we get the same result. Such a function is said to be a **pure** function. It does not depend on Rahu kaala or gunika kaala. It does not depend on the gender or the mood of the person applying it!

In programming, a function is a sequence of code. When the function is called or invoked with arguments, that sequence of code will be executed and the result is given back to the **caller**- one who invoked the function – by the **callee** – one who got called.

We use functions in programming for a number of reasons.

- Divide and conquer:
    - We cannot deal with complex problems in one go. We may want to divide the problem into small parts and solve each one of them
- sharing:
    - We can develop solutions to commonly occurring problems. These solutions can be grouped together in what is called a **library**. We can avoid reinventing the wheel.
- Maintenance:
    - A code may be required at many places. By putting in a function, we avoid redundancy of code. This helps with maintenance. Any updates need to be done at only one place.
- Reliability:
    - Functions which become part of libraries are normally well tested and are made flexible and efficient. We can use these functions with the guarantee of correctness and efficiency. We are not normally very sure of our own functions!

- Reuse
    - A function may be used repeatedly. We do not have to keep writing the function.

Function Definition:

The structure of function definition is as follows.

returntype fnname(<list of parameters with type>)

{

<stmt>

return <expr>

}

A function should have a return type. A function always returns a value of a particular type. The compiler should provide a temporary location of this type to hold the result before it is consumed by the caller. The compiler may also convert the expression of return to that of the return type if they do not match.

If a function is not required to return a value at all, then the function return type is made void.

The function name follows the rules of an identifier.

Following the function name, with in parentheses, we specify the **parameter** list. A parameter list has a list of parameters, comma separated. Each parameter is declared with respect to its type. When a function is invoked, these parameters are created on an activation record or stack frame. When the function returns, this stack frame or activation record is removed or popped out.

Parameters are always variables. They are initialized on the function call with arguments being copied to the parameters. Parameters exist only within the function call. There is no concept of closure as in Python here.

A function may have no parameters – not at all common. Even then, the parentheses are required.

The body of the function follows the parameter list.

Refer to ex0.c for some examples of function definitions and function calls.

```
int f1(int x)
{
	return x * x;
}

int f2(int x,  int y)
{
	return x + y;
}

double f3(int x)
{
	return sqrt(x);
}

void f4()
{
	printf("f4 says hello\n");
}
int main()
{
	// calling f1
	int a = 25; int b = 10;
	int res;

	res = f1(a); printf("res : %d\n", res);
	res = f1(11); printf("res : %d\n", res);
	res = f1(11 + a); printf("res : %d\n", res);
	// res = f1(int); // error

	res = f2(a, b); printf("res : %d\n", res);
```

```
    printf("sqrt : %lf\n", f3(25.0));
    printf("sqrt : %lf\n", f3(a));

    f4();
}
```

Function call:

fnname(arguments …)

A function call has the function name followed by parentheses within which we have a comma separated list of arguments. Arguments are expressions.

When a function is called, the following actions take place.

- An activation record or a stack frame is created. This contains the following.
  - Return address – a place to return when the function execution is complete.
  - Parameters
  - local variables
  - temporary variables
  - location for return value – of return type
- Arguments are evaluated. Arguments are always expressions. The order of evaluation of arguments is not defined.

  a = 10; f(a++, a); may result in any of the following calls.

  f(10, 10) or f(10, 11)
- Arguments are copied to the corresponding parameters.
- The control is transferred to the called function or the callee.
- The function body(called) is executed.
- When the function executes a return statement or reaches the end of the function body, the callee returns to the caller.
- If the function has a return statement of the form return <expr>, the value of the expression following return is evaluated and is copied to the return location.
- The caller picks up the return value. That is the value of the function call. A function call is an expression.

- If a function is a void function, the control is returned from the callee to the caller and the last two steps do not happen.

Declaration and Definition:

This is a function definition.

```
int f2(int x,  int y)
{
        return x + y;
}
```

Can we call the function before definition?

Can we call a function defined in another file?

How would the compiler know

- the number of arguments to pass
- the types of arguments to pass
- the order of arguments to pass
- the return type?

The compiler does not. Normally by default the return type is assumed to be an int. The compiler cannot check for any of these unless it knows what the function expects. We provide this information by declaring the function before calling.

```
int f2(int x,  int y);
```

This is a function declaration. It is also called **prototype** or **signature** or **specification** of the function. This concept came to C from C++.

There are two terms which we should understand clearly.

Interface and Implementation.

The function declaration is the interface. It tells us what the function expects. It does not tell us how the function works. If we change the function definition, the user or the client of the function is not affected.

```
int f2(int x,  int y)
{
        int temp = x+ y;
        return temp;
}
```

Parameter Passing mechanism:

In 'C', the argument is copied to the corresponding parameter. The parameter is not copied back to the argument. This is not possible if the argument is not a l-value.

Changing the parameter will not affect the corresponding argument.

Let us examine the code from ex2.c.

```
void foo(int x); // declaration

    int a = 10;
    foo(a);
    printf("a : %d\n", a); // 10
```

Observe that the variable a has not changed. It is impossible for the function foo to change the argument a.

```
void foo(int x); // declaration
    double b = 2.5;
    foo(b);
    printf("b : %lf\n", b); //  2.5
```

In this case, as the type of argument does not exactly match the type of parameter, the argument is cast into an integer.

```
foo((int)b);
```

But the value of the variable b is not affected either by casting or by the function call.

How to change arguments by calling function?

As the parameter passing is only by value, we may pass explicitly a pointer to an argument. Then the parameter becomes a pointer to the argument. Dereferencing the pointer gives us lvalue of the argument. By assigning to the dereferenced pointer, we can change the corresponding argument.

```
    int a = 10;
    int *p = &a; // pointer variable
    what(p);
    printf("a : %d *p : %d p : %p\n", a, *p, p);  // 100 100 ...
```

```
void what(int* q)
{
        int temp = 100;
        *q = temp;
}
```

Remember that you pass pointer to variables in scanf so that the variables are changed before scanf returns.

What if we just change the pointer? Let us look at this piece of code.

```
void bar(int* q)
{
        int temp = 100;
        q = &temp;
}
        int a = 10;
        int *p = &a; // pointer variable
        bar(p);
```

This will not change a. q gets a copy of pointer to a. The copy is changed to point to a local variable. So, clearly the variable a and variable p are not affected. If we want to change a pointer by calling a function, we should pass a pointer to a pointer. The corresponding parameter will be a pointer to pointer.

Return mechanism:

The return is always through a temporary. The expression of return is computed and copied to the temporary variable by the callee. The type of return comes into play here. If the expression of return is not same as the return type of the function, the expression is cast to the return type before copying to the temporary location. The caller will pick up the return value from this temporary and use it the way it wants.

The functions in 'C' return a single value. It could also be a pointer. Let us look at a couple of examples.

This function receives a pointer to a variable and returns the same. Not a very meaningful example. This shows how we can return a pointer from a function.

```
int* one(int* x)
{
        return x;
}
int c = 1000;
int *p = one(&c);
printf("%d \n", *p); // 1000
```

How about this?

```
int* two()
{
        int x = 111;
        return &x;
}
p = two();
printf("%d \n", *p);
```

x is a local variable. Local variables will be part of the stack frame which is created when the function is called and destroyed when the function returns. We are returning a pointer to such a variable. The pointer p in the caller will point to a variable which does not exist. This is a classical example of dangling pointer. The pointer exists; the location to which it points does not. There is no location; but we have access to it.

Dereferencing such a dangling pointer is extremely dangerous. This clearly is an undefined behaviour.

Array

Array is somewhat similar to list in Python. There are also differences.

Array has the following characteristics.

- has # of elements
- all the elements should be of the same type – homogeneous
- size of the array is specified at the point of creation
  - size is given as a constant at compile time
  - size could be specified at runtime in C99 VLA : variable length array
- array cannot grow or shrink
- elements are selected based on index or subscript
- index is an integer; starting value is 0 – zero based indexing
- supports random access
  - time required to access the element does not depend on its position
  - is always same
- Array at compile time is an array. It knows everything about itself.
- Array at runtime degenerates to a pointer.
  - At runtime, the size of the array is not stored.
- Index out of bounds – accessing elements beyond the size of the array cannot be checked.
  - Checking would require the size of the array at runtime – extra memory
  - checking would incur some runtime cost.
  - This if done would be penalizing the right programmer
- So, 'C' follows the philosophy. The programmer gets what he deserves!
- C array at runtime is a constant pointer
- So, arrays cannot be assigned one to another. Arrays are not assignment compatible.

You may read the following notes along with the file 1_array.c.

This is the way to declare an array of 5 elements and initialize.

```
int a[5] = {11, 22, 33, 44, 55};
//printf("what : %d\n", a[5]); // undefined
```

In this array, the elements are in the position 0 to 4.

We can access the element as a[2]. This can be used as r-value or l-value.

What happens if we access outside the array bounds?

What happens if we access a[5]?

There is no checking at runtime. This is an undefined behaviour. The program may crash if you are lucky. You may get anything in the world. You get what you deserve.

Array initialization:

int a[] = {10, 20, 30, 40, 50};

The compiler will count the # of elements in the initializer and make the array have so many elements. This array will have 5 elements.

int b[5];

This array will have 5 uninitialized elements when declared within a block.

int c[5] = {11, 22};

When the array is partially initialized, the remaining elements become 0.

Array Traversal:

a) using indexing

```
int a[] = {10, 20, 30, 40, 50}; int n = 5;
 for(int i = 0; i < n; ++i)
{
        printf("%d ", a[i]);
}
printf("\n");
```

b) Using pointer: version 1

As the array is a constant pointer, we can initialize a pointer variable with an array or we can assign an array to a pointer variable.

```
int a[] = {10, 20, 30, 40, 50}; int n = 5;
int *p = a;
```

or

```
int *p; p = a;
```

We can then use p wherever we use the array name a.

```
 for(int i = 0; i < n; ++i)
{
        printf("%d ", p[i]);
}
```

```
printf("\n");
```

c) Using pointer: version 2

We can also use pointer arithmetic to access the elements.

These are the valid pointer arithmetic.

- Add an int to a pointer
- Subtract an int from a pointer
- take difference of two pointers when they point to the same array.

No other operations are allowed. We should never treat pointer as an integer.

Adding an integer to a pointer advances the pointer to the ith integer from there.

```
int a[] = {10, 20, 30, 40, 50}; int n = 5;
int *p = a;
 for(int i = 0; i < n; ++i)
{
        printf("%d ", *(p + i));
        // we can also use *(a + i)
}
```

d) Using pointer: version 3

The pointer can be incremented; array cannot. Array is a constant pointer.

There is a difference between these two expressions.

(*p)++ and *p++.

In the first case, *p is incremented. The value of the expression is the old value of *p.

In the second case, p is incremented. The old value of p is stored in temporary and then dereferenced.

We can use the following code for displaying the array.

```
int a[] = {10, 20, 30, 40, 50}; int n = 5;
int *p = a;
 for(int i = 0; i < n; ++i)
{
        printf("%d ", *p++);
}
```

d) Using pointer: version 4

As the array and pointer have the same value, can we replace p by a?

```
int a[] = {10, 20, 30, 40, 50}; int n = 5;
int *p = a;
 for(int i = 0; i < n; ++i)
{
        printf("%d ", *a++);
}
```

The answer is NO. Array cannot be changed.

Passing array as an argument to a function :

When an array is an argument, the corresponding parameter is a pointer. The parameter gets the value of the array.

The parameter is not an array. It can never be an array.

We can ask the parameter whether it refers to an array. There is no way to find the number of elements in the array through the pointer.

These are the possible ways to send array as argument.

a) array and its size

b) pointer to the beginning and to the end of the array

c) have a logical element to mark the end of the array – this concept is called sentinel.

In this example, we will pass the array and its size.

```
void read_array(int x[], int n);
void disp_array(int *x, int n);
```

int x[] and int* x are same.

These are the routines to read and display the arrays.

```
void read_array(int x[], int n)
{
        for(int i = 0; i < n; ++i)
```

```
        {
                scanf("%d", x++); // &x[i]   x + i  x++
        }

}

void disp_array(int *x, int n)
{
        for(int i = 0; i < n; ++i)
        {
                printf("%d ", *x++); // x[i]   *(x + i)
        }
        printf("\n");
}
```

check these files.

2_array.h 2_util.c 2_util.h makefile

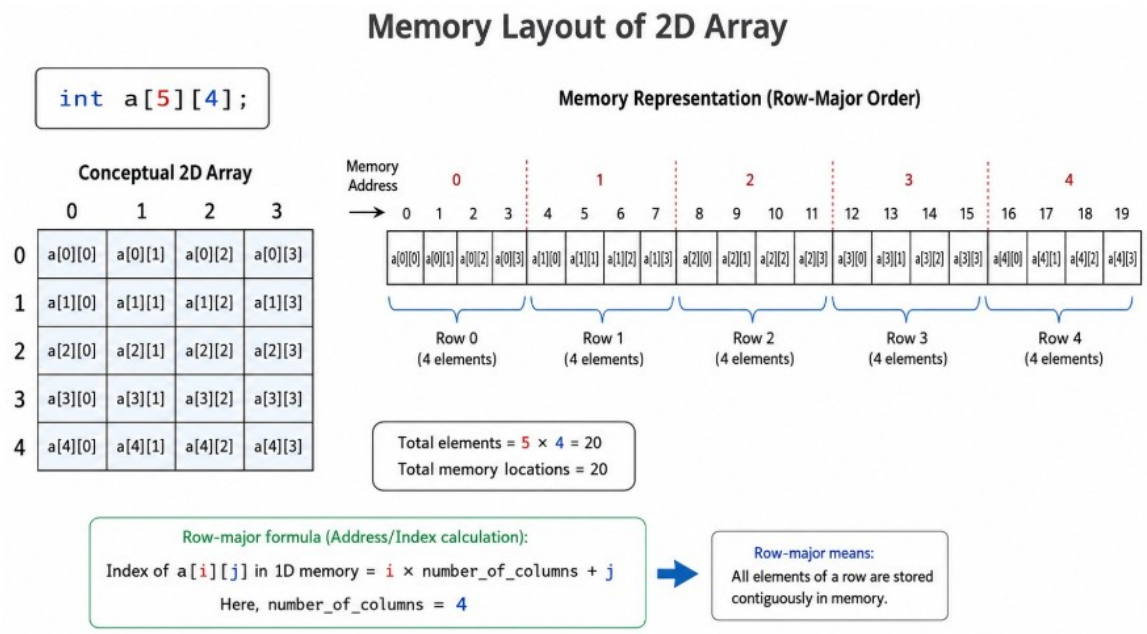
Multidimensional Arrays

Arrays are homogeneous. Every element of an array has to be of the same type. Each element itself can be an array. Such an array is called multidimensional array. Matrix is an array of arrays or is a two dimensional array. We can also have n dimensional vector spaces.

A two dimensional array is declared as follows.

int a[5][4];

The array a conceptually has 5 rows and 4 columns. The variable a is an array of 5 elements each of which is an array of 4 elements of int. Conceptually, this looks like a matrix. Internally, the elements are laid out linearly and sequentially in rowwise order. At run time, the multidimensional array just becomes a sequence of locations! The array name degenerates to a pointer to a row. // diagram



What happens when we access a[3][2]? How does the runtime find this element in the linear arrangement?

a[3] indicates that the runtime should skip 3 rows and each row occupies 4 elements – so skip 12 elements from the beginning of the array. The column index being 2 indicates, skip two more elements. So, a[3][2] will be the 14th element from the beginning of the array. Note that without the knowledge of the size of each row – the number of columns – it is impossible to locate an element in the two dimensional array.

The calculation of the location of an element in a two dimensional array stored in row major fashion is done as follows.

Address of(A[i][j]) = Base_Address +((i * No. of columns in every row) + j)*size of every element

Example: int a[2][3]={{66,90,86},{23,44,11}}
Base address is 2000 and size of integer=4 bytes
address(11)=address(a[1][2])=2000+((1*3)+2)*4 = 2000+20=2020

Read and display two dimensional arrays:

The program 1_ex.c shows how to read and display a two dimensional array. int a[5][4]; Is the declaration of the array.

int m; int n; // m : # of rows should be less than or equal to 5 and // n : # of columns should be less than or equal to 4

Reading and displaying a two dimensional array would require loop in a loop, The outer loop provides the index for the row and the inner loop provides the index for the column.

```
int main()
{
        int a[5][4];  int m; int n; // m : no. of rows; n : no. of
        col scanf("%d %d", &m, &n);

        // read the array
        int i; int j;
        for(i = 0; i < m; ++i)
        {
                for(j = 0; j < n; ++j)
                {
                        scanf("%d", &a[i][j]);
                }
        }
        // display the array
```

```
        for(i = 0; i < m; ++i)
        {
                for(j = 0; j < n; ++j)
                {
                        printf("%5d ", a[i][j]);
                }
                printf("\n");
        }
}
```

Initialize two dimensional arrays:

We can initialize the whole 2D array by initializing each row in turn.

```
int a[5][4] = {
        {1 , 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}, {17, 18, 19,
20}
} ;
```

This array is partially initialized. The rest of the elements will be 0. int b[5][4] = {{1, 2, 3,
4}, {5, 6, 7}, {8, 9}, {10}};

We may also the elements contiguously in the way the elements are stored in a row major fashion.

int c[3][2] = {11, 22, 33, 44, 55, 66}; is same as

int c[3][2] = {{11, 22}, {33, 44}, {55, 66}};

The two dimensional array is always rectangular.

int a[][3] = {{11,22,100},{33,44}};

This array has 2 rows – counted by the compiler while initializing and each row has 3 columns as indicated in the declaration. So this array shall have the size of 6 integers. We can fnd that using sizeof operator. printf("%d",sizeof(a));

Two Dimensional arrays and pointers: Let us look at this

program array_pointer.c.

```
int main()
{
```

```
int a[5][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12},
        {13, 14, 15, 16},
        {17 , 18, 19,  20}
} ; printf("%d\n", a[3][2]);
printf("%d\n", *(*(a + 3) + 2));

int (*p)[4] = a; printf("%d\n", p[3]
[2]);

}
```

This code displays the element in the 3rd row and 2nd column – counting from 0. So, this displays 15 printf("%d\n", a[3][2]);

Array name is a pointer to a row. The expression a + 3 is a pointer to the third row. *(a + 3) becomes a pointer to the zeroth element of that row. That + 2 becomes a pointer to the second element in that row. Dereference this expression to get the element in that location.

Parameter passing of a two dimensional array:

The above discussion makes it clear that when a two dimensional array is passed as an argument to a function, the corresponding parameter will be a pointer to a row. The size of each row – the number of columns – is required to compute the position of the i row away from the beginning of the array.

So, if the function foo takes the two dimensional array as argument, then the corresponding declaration can be void foo(int x[][4]); or void foo(int (*x)[4]);

Both mean the same. Recall that the parameter is never an array.

This is also fine. void foo(int x[5][4]); // first dimension ignored

All these below code are wrong.

void foo(int x[][]);

void foo(int x[5][]);

Check the fles 2_ex.c , 2_read_write.c, 2_read_write.h.

Parameter passing of a two dimensional array – making this fexible:

Observe that the size of rows is hard coded in the function declaration.

void disp_matrix(int (*x)[4], int, int);

This function can be called only on two dimensional arrays having 4 columns. How do we make this routine fexible to receive two dimensional array of any size?

Observe the programs 3_ex.c 3_read_write.c 3_read_write.h.

void read_matrix(int rowsize; int columnsize; int x[rowsize][columnsize], int rowsize, int columnsize, int m, int n);

void disp_matrix(int rowsize; int columnsize; int x[rowsize][columnsize], int rowsize, int columnsize, int m, int n);

In the parameter list, whatever follows the rightmost ; is the actual declaration of parameters. This should be matched by the argument list. Before the actual declaration of parameters, we have forward declaration of parameters - telling the compiler that there are parameters – (rowsize and columnsize) which appear later in the parameter list. So, we are now allowed to declare the parameter standing for the two dimensional array with variable sizes.

Please do observe that the columnsize specifed is required and should match the physical size – as per the declaration of the two dimensioal array and not logical size that you may programmatically choose to use.

You may try a few interesting programming assignments.

1. addition of two matrices
2. multiplication of two matrices
3. generate magic square
4. generate Pascal triangle
5. fnd the biggest element
6. search for an element

…

Structures

Most of you carry a backpack. It would contain laptop, some books, pens, pencils, water bottle, some snacks and chocolates. How would it be to carry all these without a bag?

In programming, we encounter many cases where we may have related items – not necessarily of the same type. It will be easier to refer to this collection by a single name.

Structure : an example:

A structure is a named collection of items not necessarily homogeneous.

Refer to the program 1_structure.c.

```
struct student
{
        int rnum;
        char name[20];
        int marks;
};
```

This declaration says that the struct student is a structure which contains 3 elements – called fields or data members. These are rnum - of type int, name - an array of char and marks - an int.

The struct type declaration decides the binary layout of the structure. It decides the order of the fields and the total size of a variable of that type. It does not create any variable – so takes no memory. It is only for the compiler. As we will see later, this should be part of the header file – as it is required for both the client and the implementor.

Do observe the ; at the end of the block. This is required to indicate that no variables are declared following the type declaration. We could in fact declare variables following the } before the ; .

The following code creates a variable of struct type.

```
struct student s1;
```

We can access the fields of this structure variable using . Operator.

```
strcpy(s1.name,"abc");
s1.marks=78;
s1.rnum=56;
```

We can also create a pointer to a structure as follows.

struct student *p = &s1;

We can access marks of struct student pointed to by p in the following two ways.

(*p).marks

p is a pointer to structure; *p is a structure; require parentheses to take care of precedence

or

p→marks

p is a pointer to structure; p→ is a structure dereferenced already; do not require to access the field

characteristics of a structure:

A structure has one or more components

- components are generally heterogeneous
- components have names - structure has named components
  - in case of array, the components are computed based on the value of an expression
- offset to each component is fixed at compile time
- supports random access. Time required to access the component does not depend on its position in the layout of the structure.
- The size of a structure depends on implementation. The size of structure is at least equal to the sum of the sizes of data members. We shall discuss this with a few examples later in this chapter.
- A structure variable stands for the whole structure. It does not degenerate to a pointer as is the case with arrays.
- Structures are assignment compatible. Conceptually, the compiler generates the code for memberwise copy. Structures can be assigned even if the structure contains an array.

structure layout and size:

check the file : 2_structure_size.c.

Let us consider one of the examples from this file.

```
struct test1
{
        char j;
        int i;
```

};

How big is this structure if char takes one byte and int takes 4 bytes on a particular implementation? The answer is simple. It depends on the implementation.

One of the possible answers is 5. This is obtained by adding the sizes of the components.

Let us take an analogy. While making notes in the class, would you start the next topic on the same page if it is partially full or would you go to the next page? The decision is yours. If you start in the same page, you save space. If you start on the next page, searching may become faster.

In 'C', every datatype has an alignment criterion. The value of a type may start on a byte boundary, word boundary, paragraph boundary and so on. The size of an int is normally size of a word. If the int field is word aligned, accessing the int field will be faster.

So structures could align for each of the types and there could be some space in between the fields which is not used – this is called padding.

This structure may report 8 bytes – then this is efficient in time or it may report 5 bytes – then this is efficient in space.

Check the file: 3_struct_pack.c.

#pragma pack(1)

We can change the alignment criteria of types by using this compiler directive. pragmas are compiler directives. In general, pragmas may not be available all compilers – so we say that they are not portable.

This program says align every type to a byte boundary – no padding.

Run the program with various values of pack and convince yourself about the sizes.

Initialization of variables:

Refer to the program 4_structure_initialization.c.

This structure s1 is not initialized. So, the fields are uninitialized.

```
struct student s1;
printf("no initialization done explicitly---\n");
// some junk displayed
printf("%d\n",s1.rnum);
printf("%s\n",s1.name);
printf("%d\n",s1.marks);
```

The structure s2 is initialized. Observe the order of values specified in the initialized matches the order of fields in the structure.

```
struct student s2={23,"pes",99};
printf("%d\n",s2.rnum);
printf("%s\n",s2.name);
printf("%d\n",s2.marks);
```

In this case, s3 is partially initialized explicitly and the rest are filled with 0.

```
struct student s3={23,"pes"}; // creation of structure variable and initialization together. But only few members are initialized explicitly. others are initialized to zero
printf("%d\n",s3.rnum);
printf("%s\n",s3.name);
printf("%d\n",s3.marks);
```

In this, the fields are initialized by specifying their names in the initializer. This is called designated initialization.

```
struct student s5={.name="pes",.marks=45};                    //
printf("%d\n",s5.rnum);
printf("%s\n",s5.name);
printf("%d\n",s5.marks);
```

Copying structure:

The structures get copied either in initialization or in assignment. Conceptually, the corresponding fields are copied – called memberwise copy.

Let us examine the file: 5_struct_memberwise_copy.c

```
struct test1
{
        int i;
        char ch;
};
struct test1 t1={23,'w'};
struct test1 t2=t1;
```

t2.i will be 23 and t2.ch will be ‘w’.

But t2 does not share memory with t1.

We have no operators to compare structures directly. We can compare the corresponding fields if they can be compared.

```
if(t1.i==t2.i && t1.ch==t2.ch)
        printf("equal");
else
        printf("not equal");
```

This should display equal.

```
t1.i=189;
```

This statement shall not affect t2.i.

If the component of the structure is a pointer, then we may get unusual issues. We shall discuss them later.

Parameter passing:

A structure is passed by value – a copy of the argument is passed to the parameter. Observe the program 6_structure_parameter_passing.c.

```
struct test
{
        int i;
        char a[20];
        float j;
};
void f1(struct test t)
{
        strcpy(t.a,"pqrstuv");
}
struct test t1={20,"xyz",78.5f};
printf("%s",t1.a);
f1(t1);
printf("%s",t1.a)
```

t gets a copy of t1. Change to a of t does not affect a of t1.

In case, the fields of the structure need to be changed by calling a function, pass pointer to the structure as argument and the corresponding parameter will be a pointer to a structure.

```
void f2(struct test* p)
{
        strcpy(p->a,"pqr");

}
f2(&t1);
printf("%s",t1.a); // pqr
```

Passing a structure by value is considered a bad programming practice. This would occupy more space, would require more time to copy and also will cause problems if the members of the structure are pointers.

When we want to change the structure, pass a pointer to a structure as argument.

When we do not want to change the structure, pass a pointer to a structure and make the parameter a pointer to a constant structure.

This function f3 shows how to pass a structure if we do not intent to change the structure.

```
void f3(const struct test* p)
{
        //strcpy(p->a,"pqr"); // Error
        printf("%s\n", p->a);

}
f3(&t1);
```

return from a function:

A function can return a structure or a pointer to structure. In the latter case, the pointer should point to non-local variable.

Observe the program : 7_struct_return.c.

```
#include<stdio.h>
#include<string.h>
struct test
```

```
{
        int i;
        char a[20];
        float j;
};
struct test f1(struct test t);
int main()
{
        struct test t1={20,"xyz",78.5f};
        printf("%s",t1.a);
        t1=f1(t1);
        printf("%s",t1.a);

}
```

This function returns a changed parameter by value. This is copied to a temporary return object. Then in the client code, the temporary return object is assigned to t1.

```
struct test f1(struct test t)
{
        strcpy(t.a,"pqrstuv");
        return t;
}
```

typedef : creating an interface:

Whenever the client creates a variable a of integer type, he uses the following code.

```
int a;
```

He does not know how the variable is stored and does not have to know how it is stored. He should know what he can do with it – in terms of operations like addition, subtraction and so on.

But, when the user makes a variable struct student, he gets to know that this is a structure. He should not directly play with the structure as changing would become difficult. So, we would like to give him a single name for the type and by convention, he should do nothing directly with the structure. We may change this to an array or structure of structures or whatever which makes sense for us without affecting the functions the client can call.

We do that by using a construct called typedef. It gives a name for an existing type. We will discuss more of it later in the course.

typedef struct student student_t; // creates a typename.
student_t s1; // creates a variable.

```
#include<stdio.h>
struct student
{
	int rnum;
	char name[20];
	int marks;
};
typedef struct student student_t; // creates a typename

int main()
{
	student_t s1; // creates a variable
	// ...

}
```

Dynamic Memory Management

So far, we have used variables which are created when the block is entered and which are removed when the block is exited. We have no control over the creation and destruction of variables.

Is it possible to get memory whenever we require? Is it possible to relinquish the memory when we no more require? Can we get memory which is not bound to any particular type and then we can create variables of a particular type within that memory?
It is possible. We use a technique called dynamic memory management to achieve this.

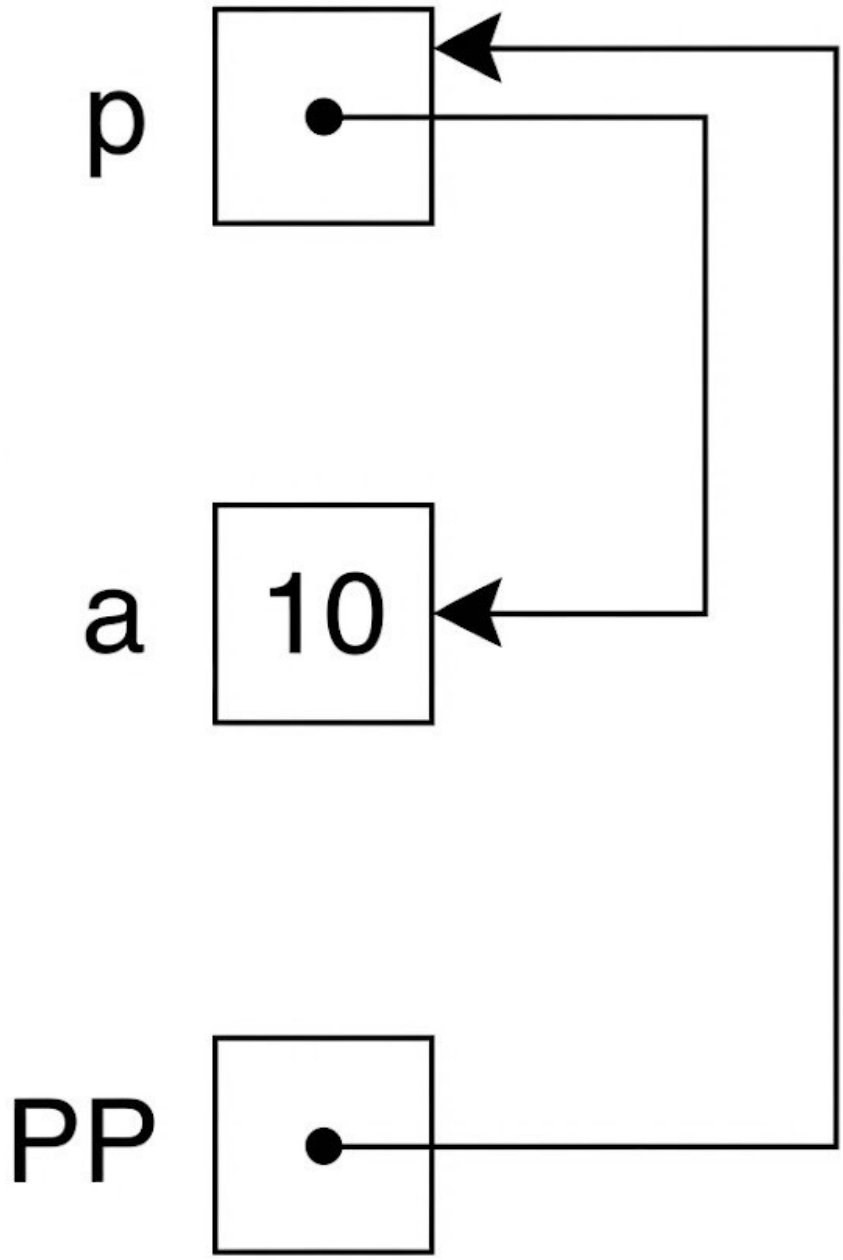
Before we get into this topic, let us revise the concept of a variable.
A variable in a block is created on entry and is not initialized by default. A variable uninitialized or otherwise has a location – therefore an address.
If a variable is of pointer type, we should initialize the variable to point to a variable – dereferencing before initialization is clearly an undefined behaviour. Observe the program given below.

```
// 1_ex.c
#include <stdio.h> int
main()
{
#if 0
        int *p;
        printf("&p : %p\n", &p); // ok
        printf("p : %p\n", p); // logically wrong; using an uninitialized variable
        // program will not crash
        printf("*p : %d\n", *p); // terrible code; looking content of an uninitialized variable
        // as a  pointer and dereferencing it; could crash; undefined behaviour #endif
        int *p;
        int a = 10; p
        = &a;
        printf("&p : %p\n", &p); // ok
```

```
    printf("p : %p\n", p);  // ok; p has been assigned a value
    printf("*p : %d\n", *p); // same as a; ok if a has been given a value

    int **pp;
    //pp = & &a; //1_ex.c:21:7: error: lvalue required as unary ‘&’ operand pp = &p;
    printf("&pp : %p\n", &pp);
    printf("pp : %p\n", pp);
    printf("*pp : %p\n", *pp);
    printf("**pp : %d\n", **pp);

}
```



Also, observe that if a is a variable, &a is a pointer to it and it is a r-value. & & a is a guaranteed compile time error as &a is not a lvalue – cannot appear to the left of assignment.

The variable pp is a pointer to a pointer by type – can point to a variable which can point to an int.

Clearly understand what these expressions stand for.
&pp : pointer to a variable whose type is pointer to pointer to int pp :
variable whose type is pointer to pointer
*pp : expression(both l and r value) which points to an int
**pp : expression (both l and r value) which is an int

There are no operators to support dynamic memory management in 'C'. There are functions using which we can effect dynamic memory management.
This logical memory area used by dynamic memory manager is called heap.

malloc : allocates memory of a given size and returns a pointer free :
returns the memory allocated back to the memory manager calloc :
allocates memory and initializes the memory with 0
realloc : reallocates memory; copies the old values into the new memory locations. Please check the man pages for these functions.

```
// 2_ex.c
#include <stdio.h>
// dynamic memory management : malloc and free allocate and deallocate memory on the heap
int main()
{
#if 0
        int *p;
        // request for memory
        //      function : malloc requires the # of bytes to be passed as argument
        // returns a generic pointer : void*
        // convert pointer to void to pointer to int
        // creates a variable with no name; can be accessed using a pointer p =
        (int*)malloc(sizeof(int));
        *p = 20;
        // return the location back to runtime : free

        free(p);
#endif
```

```
#if 0
        int *p;
        p = (int *)malloc(sizeof(int) * 4); // array on the heap p[0] =
        11; *(p + 1) = 22;
        free(p);
// on allocation of memory, malloc stores the size of memory allocated in some location
// That is based on implementation.
// Given the pointer value returned by malloc, it is possible for the implementation
//      to find the size allocated
// the function free finds the book keeping or house keeping info and deallcoates the
//      required amount of memory
#endif

        int *p;
        p = (int *)malloc(sizeof(int) * 4);

        free(p + 1); // TERRIBLE thing to do
        // p + 1 was not returned by malloc
        // there is no meaningful book keeping associated with p + 1
        // anything may happen
        // UNDEFINED BEHAVIOUR
}
```

Let us examine the program 2_ex.c.

int *p;

At this point, we have the variable. The variable p has not been initialized. Dereferencing p at this point is criminal.

p = (int*)malloc(sizeof(int));

We ask malloc to give us as many bytes as an int requires. Observe we do not hard code the # of bytes an int occupies as it depends on implementation.

The function malloc returns a generic pointer – void*.

If the allocation fails, malloc returns NULL. We have then plan what to do in such a case. We shall such examples later.

We convert that to pointer to int by casting.

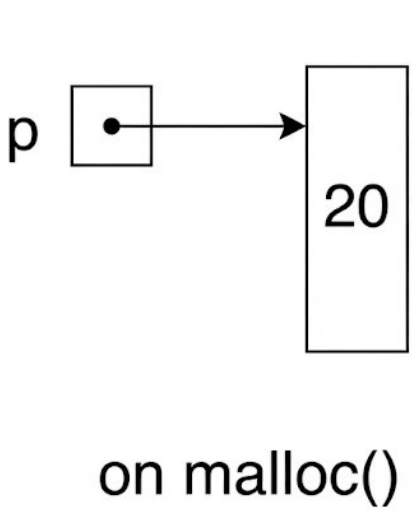
*p = 20;

We assign 20 to the int to which p points to. Observe that, this variable does not have a name.

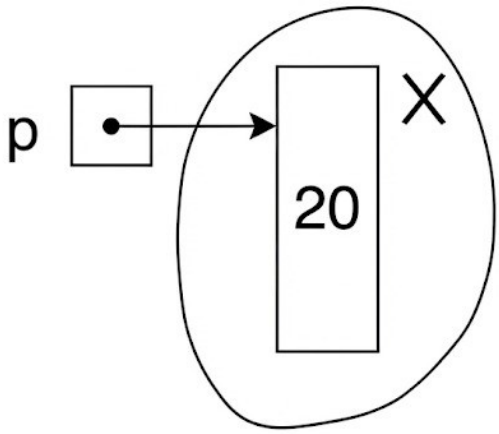
// return the location back to runtime : free

free(p);

Gives the location back to the runtime management. Observe as the parameter passing is by value, the value of p cannot change when free is called. It will still have the old value.



on malloc()



on free()

Let us consider another example of memory management. int *p;

p = (int *)malloc(sizeof(int) * 4); // array on the heap

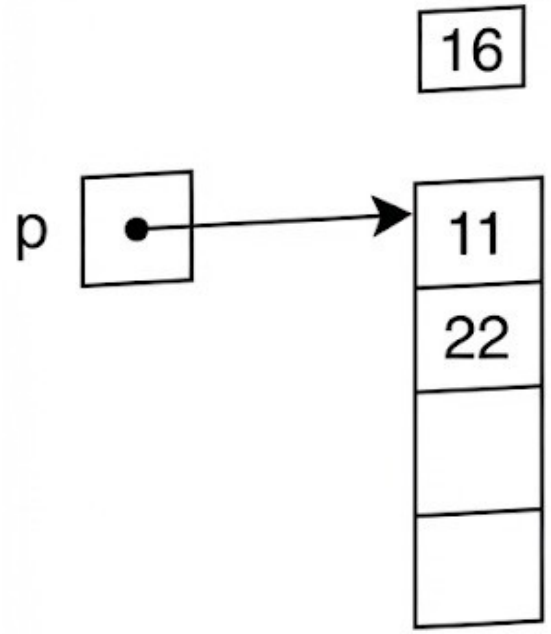
The above statement causes allocation of memory for 4 integers. Once allocated, p points to an array of 4 integers. We say that p is an array on the heap.

p[0] = 11; *(p + 1) = 22;

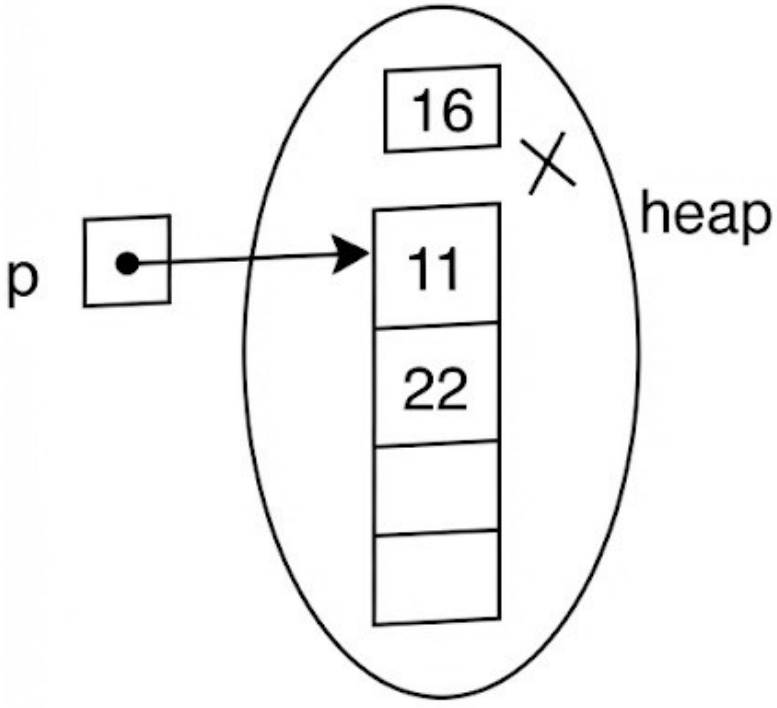
free(p);

The function free releases the whole block.

// Book keeping / house keeping



On allocation of memory

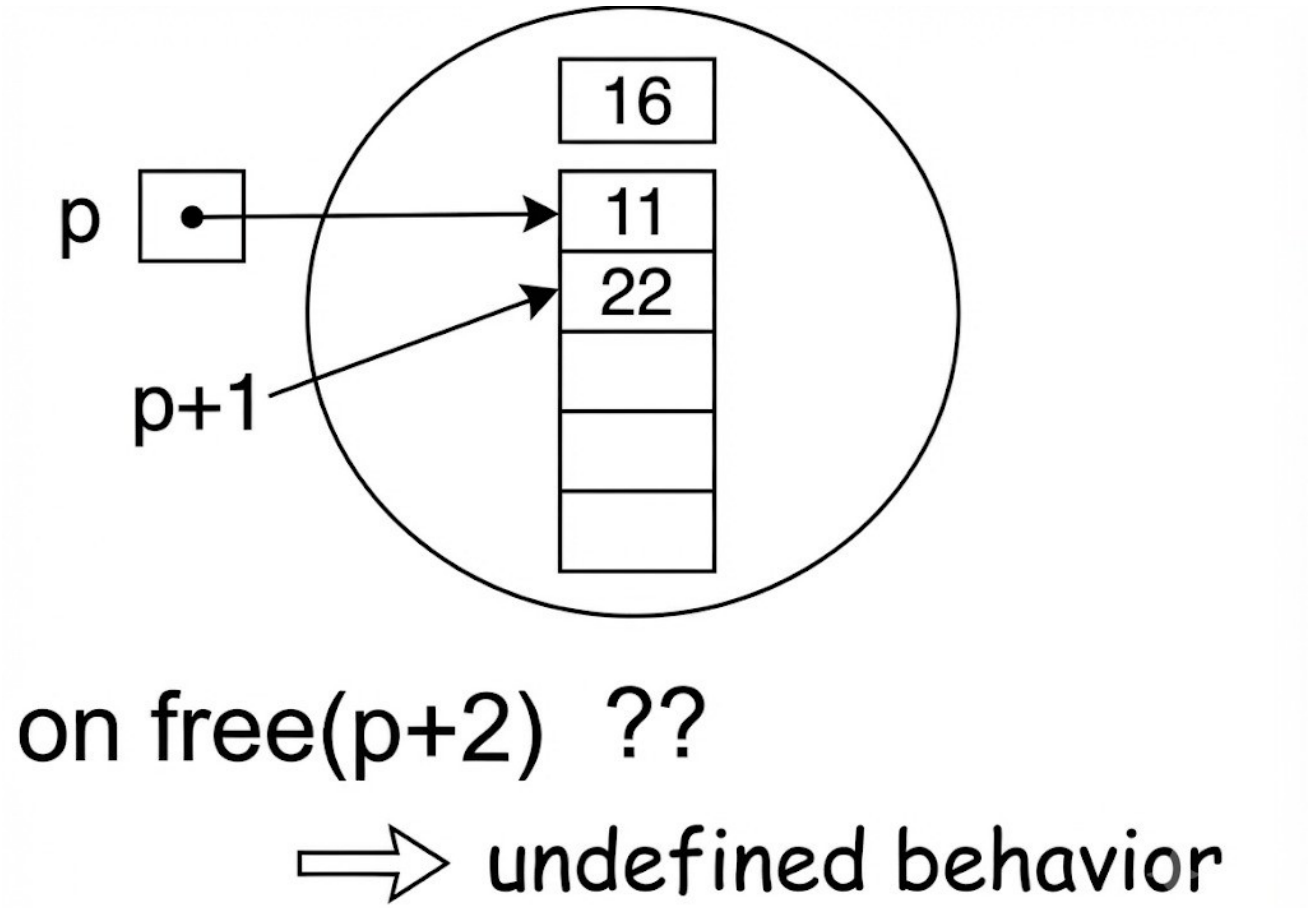


On free

You may wonder why free is not given an additional argument indicating the size of the block allocated. You may wonder how come free is able to free a single int location in the first case or an array of int in the second case.

These is an interesting concept called book keeping or house keeping. When malloc allocates memory, it remembers somewhere in memory the # of bytes allocated. This additional information maintained is called book keeping or house keeping. The function malloc returns a pointer which can be used to access the location where the # of bytes is stored. We do not know where the book keeping is kept, but the implementation knows it. When we call free with a pointer, free accesses the book keeping and finds the # of bytes to deallocate and deallocates so much of memory.

What happens if free is called on a pointer which does not point to heap?
What happens if free is called on a pointer value not returned by a previous malloc call?
What happens if free is called second time?
All these result in undefined behaviour.
We can delete locations only on the heap. The pointer should be bound to a location where book keeping is kept. Double deletion is an undefined behaviour.




```
int *p;
p = (int *)malloc(sizeof(int) * 4);

free(p + 1); // TERRIBLE thing to do; no booking keeping associated with p + 1.
```

Let us now examine a couple of issues with dynamic memory management. The program is 3_ex.c.

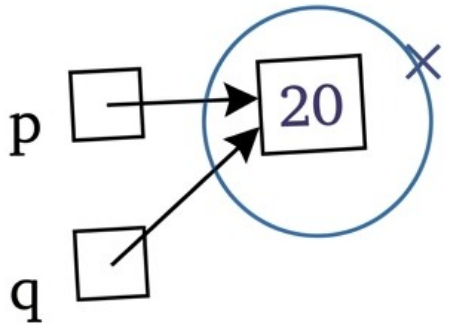
```
    int *p;
    p = (int*)malloc(sizeof(int));
    *p = 20;
    int *q = p;
```

In the

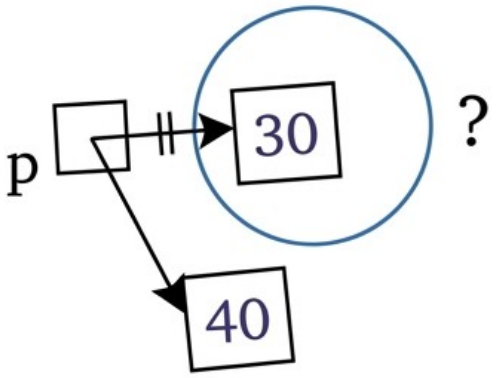
free(q); // valid statement; ok; what matters is the value of the pointer and not the name

Here, we allocate memory to p, initialize q to get a copy of p. So both p and q point to the same location – they are aliases. Can we call free(q) safely. The answer is YES. What matters is the value of the pointer and not the name. The value of the pointer is associated with book keeping. So, free can find the size allocated through the value of the pointer and release so much of memory.

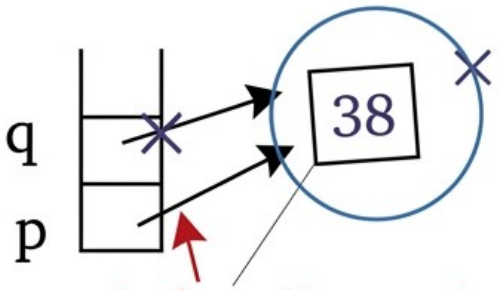
Stack , heap



on free(q)



garbage ; location with no access



* dangling pointer ; no location ; accen ;

```
// 3_ex.c
#include <stdio.h> int
main()
{
#if 0
        int *p;
        p = (int*)malloc(sizeof(int));
        *p = 20;
        int *q = p;
        free(q); // valid statement; ok; what matters is the value of the pointer  and not
the name
        // ??
? #endif


#if 0
        int *p;
        p = (int*)malloc(sizeof(int));
        *p = 30;
        p = (int*)malloc(sizeof(int));
        // changing of p makes us loose the pointer to the location allocated by
the earlier
        //      malloc  call


        // creates garbage; we have a location but no access
        *p = 40;
        // no garbage collector (python does based on reference count)
        // garbage in turn becomes a memory leak

#endif
        int *p;
        {
                p = (int*)malloc(sizeof(int));
                *p = 30;
                int *q = p;
```

```
        free(q); // cannot change q; parameter passing by value
        // q = NULL: or q = 0; // some people think it is safe programming
        // dereferencing a NULL pointer is a guaranteed crash
    }
    // p is a dangling pointer; no location; access;
    printf("*p : %d\n", *p);
}
```

Garbage and Memory Leak:

```
p = (int*)malloc(sizeof(int));
*p = 30;
p = (int*)malloc(sizeof(int));
```

Observe the code above. We allocate memory and p points to it. The second allocation will overwrite p. We loose access to the first allocated memory as it has no name and only access was through p, but p has changed now.

This integer location – first allocated memory - cannot be accessed. It is called garbage. Garbage is a case where a location exists, but there is no access to it. In some languages, there are mechanisms to make the garbage reuseful – this is called garbage collection. In Python, we have garbage collection based on reference count.

'C' does not support garbage collection as it is costly in terms of space and time. The onus of keeping memory clean is on the programmer and not the 'C' runtime. So, garbage in 'C' in turn becomes memory leak.

Dangling pointer:

```
int *p;
p = (int*)malloc(sizeof(int));
*p = 30; int
*q = p;
free(q);
printf("*p : %d\n", *p); // ????
```

In the example above, p gets allocation through malloc. The unnamed location pointed to by p is assigned. Both q and p point to the same location. Then q is freed. Now p still points to the location which no more logically exists. This is a case where we have no location, but we have access.

Both p and q are dangling pointers – access but no location.

Please note the following.

Dangling pointers are not dangerous – dereferencing them is dangerous. Dangling pointers can happen anywhere, but memory leak can only happen in the heap.

Allocation with initialization:

The function calloc allocates memory and initializes the locations with 0. Observe that calloc takes two arguments where malloc takes 1. The calloc does not give us any choice regarding initialization – it always each byte to 0.

```
// 4_ex.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
        int *p;
        //p = (int *)malloc(sizeof(int) * 4); p =
        (int *)calloc(4, sizeof(int)); printf("p :
        %p\n", p);
        for(int i = 0; i < 4; ++i)
        {
                printf("%d ", p[i]);
        }
        printf("\n");

        // change the array size?
        // increase the size:
        //              may get the same sequence of locations
        //                                      different sequence of locations
        //              old pointer is conceptually dangling
```

```
        //              values are copied; old location freed if we get a new block of
locations
        // decrease the size
        //              values at locations which remain are kept int
        *q = (int *)realloc(p, 10000);
        printf("p : %p q : %p\n", p, q);

        free(q);
}
```

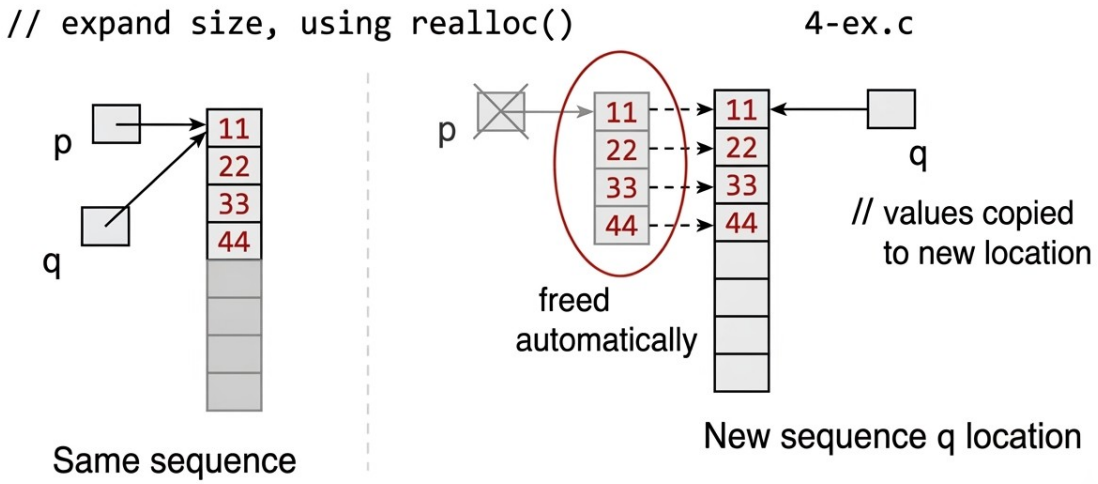
Reallocation:

```
int *q = (int *)realloc(p, 10000);
```

If p points to a block of locations on the heap with some size, the size can be altered by reallocation.

A few points to note:

- size can be increased or decreased
- the pointer returned by realloc may not be same as the earlier value. So, allocation may happen in some other part of memory
- on increase in size, all old values are copied and the rest of the locations remain uninitialized.
- On decrease in size, only the values of those locations which find new locations get copied. The rest are lost.



Allocation of structure:

```
// 5_ex.c
#include <stdio.h>
#include <stdlib.h>

struct What
{
        int *p; int
        a;
};
typedef struct What what_t; int
main()
{
// case 1:
#if 0
        what_t x; what_t
        y;
```

```
int z = 10000;
```

```
        x.a = 100;
        x.p = &z;

        printf("x : %d %d\n", x.a, *(x.p)); // 100 10000 y =
        x;
        printf("y : %d %d\n", y.a, *(y.p)); // 100 10000
        x.a = 200; *(x.p) = 20000;
        printf("y : %d %d\n", y.a, *(y.p)); // 100 20000 #endif
// case 2:
#if 0
        what_t x; what_t
        y;

        x.a = 100;
        x.p = (int *)malloc(sizeof(int));
        *(x.p) = 10000;

        y = x;
        printf("y : %d %d\n", y.a, *(y.p));
        free(y.p);
        // x.p : ? dangling #endif
// case 3:
#if 0
        what_t x; what_t
        y;
        x.a = 100;
        x.p = (int *)malloc(sizeof(int));
        *(x.p) = 10000;

        y = x;

        y.p = (int *)malloc(sizeof(int)); // ?? safe
```

```
#endif
// case 4:
#if 0
        what_t x; what_t
        y;
        x.a = 100;
        x.p = (int *)malloc(sizeof(int));
        *(x.p) = 10000;

        y = x;

        x.p = (int *)malloc(sizeof(int)); // ok #endif
// case 5:
        what_t x;
        x.a = 100;
        x.p = (int *)malloc(sizeof(int));
        *(x.p) = 10000;
        x.p = NULL;
}
```

In case 1, observe that the assignment of structures causes the pointer fields to become aliases.

x.p and y.p both point to z. Changing z through any one of these pointers affects the other.

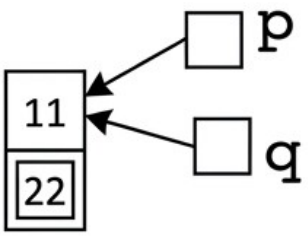
In case 2, the pointers in both x and y point to a location on the heap. Freeing through one of these pointers, make all these pointers dangling.

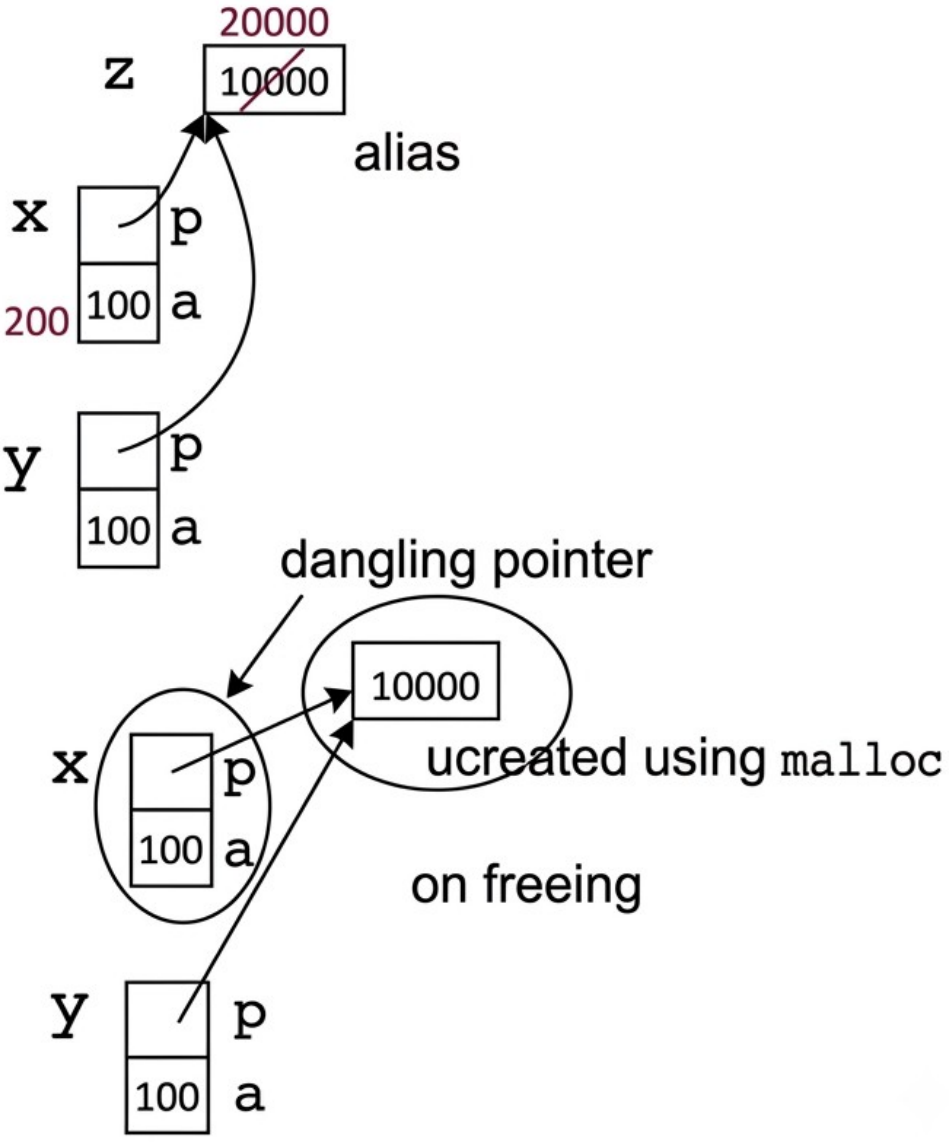
In case 3, after the assignment of x to y, both the pointers point to the same location on the heap. When y.p is assigned a new pointer value, the aliasing is broken. Both x.p and y.p point to two different locations all together – perfectly safe.

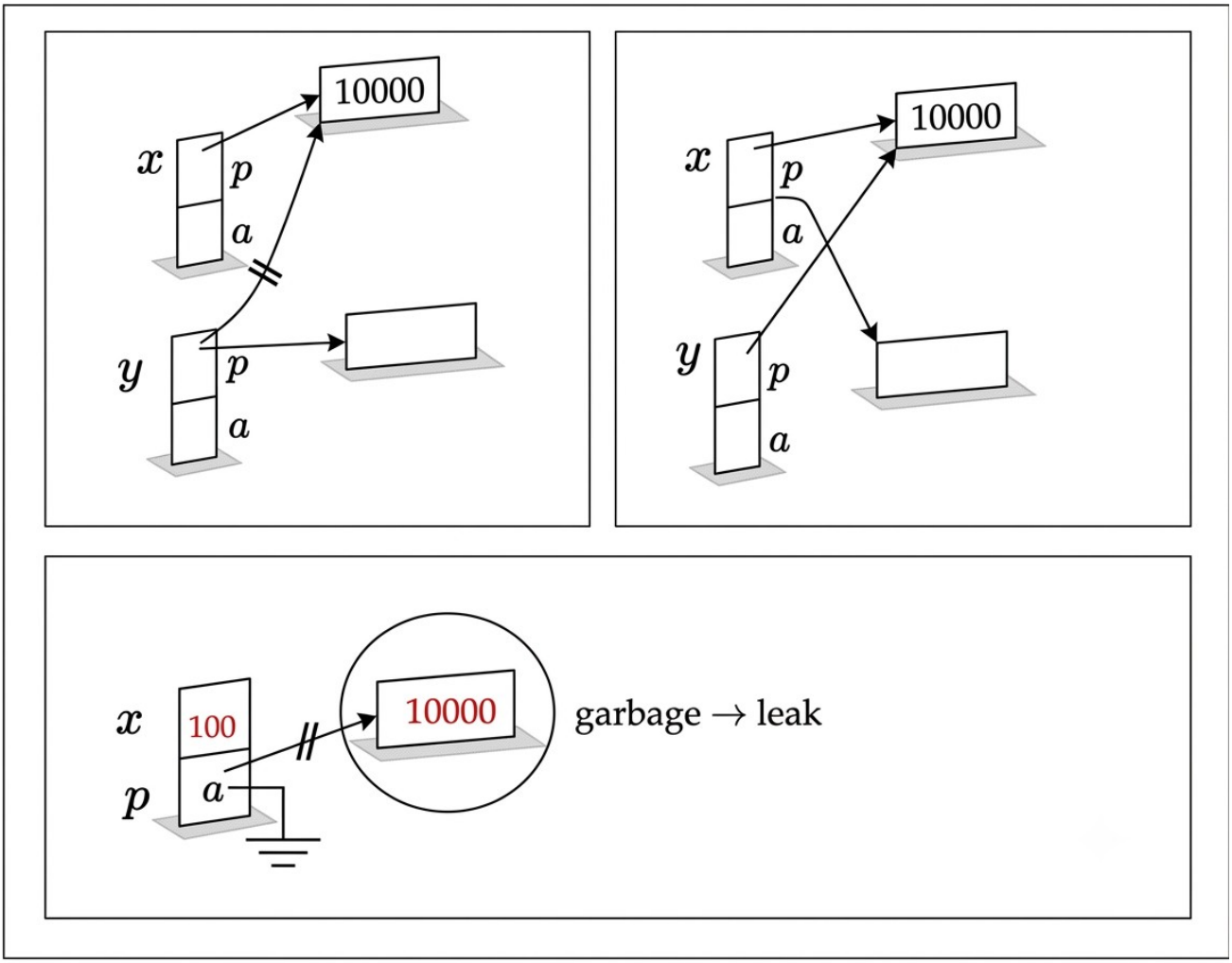
In case 4, after assignment of x to y, both the pointers point to the same location on the heap. When x.p is assigned a new pointer value, the aliasing is

broken. Both x.p and y.p point to two different locations all together – perfectly safe.

In case 5, the pointer is grounded resulting in loss of access to location on the heap – causing garbage and therefore memory leak in ‘C’.


p
11
22
q


20000
z
10000
alias
x
p
200
100
a
y
p
100
a
dangling pointer
10000
x
p
100
a
ucreated using malloc
on freeing
y
p
100
a


10000
x
p
a
y
p
a
10000
x
p
a
y
p
a
x
100
p
a
10000
garbage → leak

Concept of Type

We tend to classify our friends. Is it not? We will say that fellow is a “kudumi”. We classify a few as sportsmen. A few as musicians and so on. All those in the same group exhibit some common behaviour.

This concept of grouping has come from Mathematics – concept of Set. We study number line. We start with set of natural numbers. Apply some operations on it. Show that some do not result in a member of the same set – we say that this is set is not closed for this operation. We then extend the set – to set of integer, then set of rational numbers – the set of irrational numbers and then set of real and last set of complex numbers. My teacher had told me in my PU days that there is a theorem which says that there can be no number system beyond the complex number set.

Let us look at set of integers in Mathematics.

It is closed for addition and multiplication
25 + 34 is 59
How do we add?

- Add unit digits and then digits in tenth place and so on along with the carry from the earlier stage
- Add row wise
- Use fingers and toes
- Use tally marks
- Convert to some other base, add and convert the result back to decimal
- Use Vedic Math or Trachtenberg method

Will the result be different?
Should we learn how to add if the integers have different values?

The beauty of this grouping is that we can treat them uniformly and we can the define the results of operations on them without explicitly saying how to carry these operations.

- An integer is a type
- Represented in a programming language normally as a subset
- A language will specify the smallest and the biggest integers in its implementation.
- **A type indicates**
    - **What values are legal**
    - **What we can do with them**
    - **Does not say how we can do them**
    - **Allows us to treat almost all values of that type uniformly**

- Type of a variable in ‘C’ has to be declared in the beginning
- Once declared, cannot be changed.
- Type of a variable is determined at compile time
- In Python,
    - type is associated with a value.
    - A variable gets a type based on type of value assigned.
    - Type is dynamic.

Can a language support all possible types of the real world?

- A car is a type.
- Supports the following operations
    - Start
    - Stop
    - Accelerate
    - Brake
    - Steer
- Once you have learnt driving a car, you can (almost) drive any other car
- Other types of real world : fan, mobile phone, browser, laptop, novel, dream, ...
- Can a language support all possible types of the real world?

The answer is a big NO.

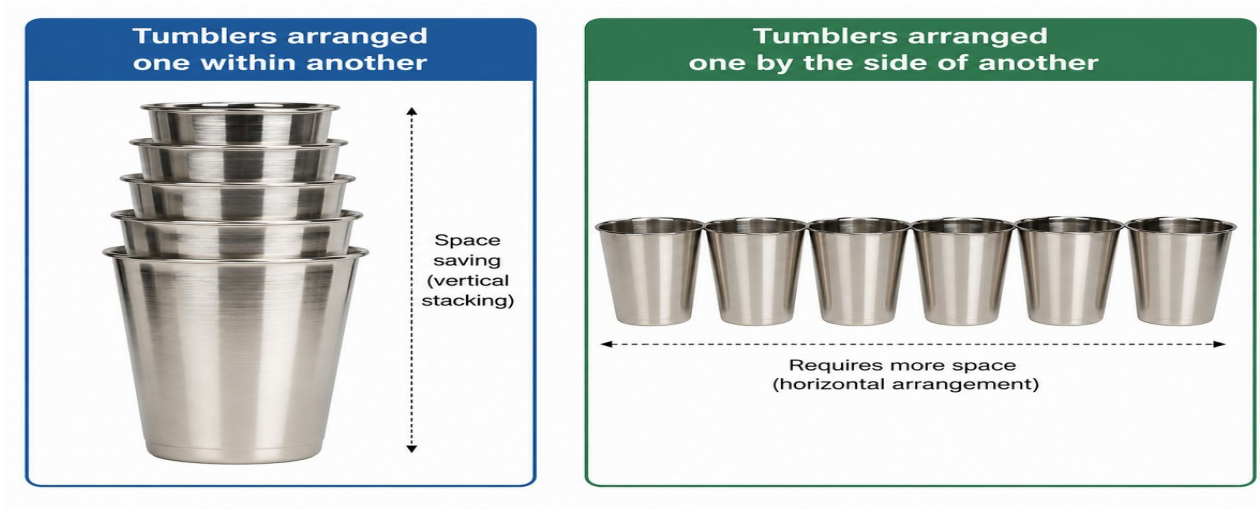
An interface is sacred – cannot be changed
An implementation – can be changed .
In a car, the braking mechanism can be changed from mechanical to hydraulic system.
Steering can be changed from manual to power steering.

Can we change the interface? When you press the brake, the vehicle accelerates? When you turn the steering to the left, the vehicle goes to the right? Can you imagine what could happen?

What should the language support?

- A Language should support a way of creating our own type
- **Abstract Data Type (ADT)**
    - Like a type (int, double, …) in a language, this supports
        - A few operations
        - Specifies what they do and not how they do
        - **Specifies the interface and not the implementation**
        - Both the user and the implementor depend on the interface
        - Interface once decided is sacred; cannot be changed
        - Implementation can be changed without affecting the user
        - This is called inversion of dependency
    - If A depends on B, any change to A would require change to B.
    - If both A and B depend on what does not change, change to A will not affect B.

Data Structure:
Is an interface
Has number of elements arranged in some particular way to support a few operations efficiently
Is constrained by time and space.



In the above example, tumblers arranged in two different ways. None of them is wrong. The first one is saving space and only one person can pick up a tumbler at a time. The arrangement on the right requires more space, but more than one access at the same time.
Should the tumblers be placed mouth up or mouth done. I do not know have an answer!

In this arrangement of books, one of the left allows easy access to any book whereas the one on the right requires that you have pull the book with a lot of effort if it is towards the bottom. The second may save space and allow more books to be placed.



None of them is right or wrong. You may select one of these arrangements based on your requirement.

List :

- Is a data structure
- Has zero or more components
- Each element has an ordinal position
- Cannot directly access the element based on position
- Sequential access
- No random or direct access
- Insertion and deletion once the position is known requires
- A few operations

- **Interface** :
    - Create
    - Destroy
    - Insert
    - Delete
    - …
- **Implement** using
    - A linked list
    - An Array

Dynamic Data Structure : linked list

We have discussed dynamic memory management using malloc, free and other functions in the last notes. We will continue from where we left and we will discuss some examples of use of dynamic memory management.

In these examples, models or diagrams are very necessary to assist us in understanding the concepts. Please draw the diagrams for yourself while reading these notes and understand how these programs work. It is extremely important to understand where the pointers point to and how they get changed.

Let us examine the code in 1_ex.c.

struct X is a type. struct Y has a variable of struct X within it. It is a classical

example of structure within a structure.

There is an issue with struct Z.

When the compiler compiles the struct defnition, it decides the **binary layout of the structure**. It decides the relative position or offset of each field and also decides the total size of the structure – with or without padding based on the **alignment** criteria.

```
struct Z
{
        int c;
        struct Z z; // infnitely recursively included
};
```

We cannot fnd the size of struct Z as we cannot fnd the size of its field z which itself is a structure of the same kind. This results in a compile time error.

Struct A is perfectly alright. The size of pointer to any type on a given implementation is fixed. So size of the structure can be computed by the compiler at compile time.

A structure which has a pointer to itself as a field is called a self referential

structure.

```
struct A
{
        int c;
        struct A *p;
};
```

// 1_ex.c

```
#include <stdio.h>
#include <stdlib.h>

struct X
{
        int a;
};

struct Y
{
        int b;
        struct X x;
};

#if 0
struct Z
{
        int c;
        struct Z z; // infnitely recursively included
};
#endif

// self referential structure
struct A
{
        int c;
        struct A *p;
};
```

```
int main()
{
        printf("size : %lu\n", sizeof(struct X));
        printf("size : %lu\n", sizeof(struct Y));
        //printf("size : %lu\n", sizeof(struct Z));
        printf("size : %lu\n", sizeof(struct A));
        struct A one;
        one.c = 10;
        one.p = &one;
        printf("%d %d %d\n", one.c, one.p->c, one.p->p->c);
}
```

In this example, pointer p of the variable one points to variable one.

So, one→p is same as one→p→p !!.

Linked List: a simple example:

A List is a data structure like an array. It has the following characteristics.

Has zero or more components

Each component has an ordinal position. We can refer to an element by its position. You may recall that set of Python does not support this concept.

We can access element one after another. There is no way to access the element directly as in the case of arrays. Arrays have random or direct access whereas lists have sequential access.

Insertion and deletion in a list at a given position requires a very few operations and requires no shifting.

A Linked list is an implementation of list using self referential structures.

Let us discuss the following piece of code.

```
struct node
{
        int key_;
        struct node *next_;
};
typedef struct node node_t;
```

So, struct node has a field called link whose type is a pointer to the same

structure. struct node is a classical example of self-referential structure.

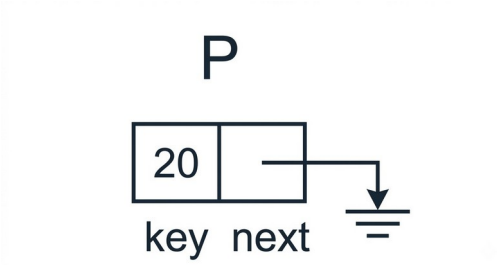
p is a pointer to structure.

node_t *p;

p = (node_t*)malloc(sizeof(node_t));

p->key_ = 20;

p points to a structure dynamically created. p→key_ has been assigned a value.



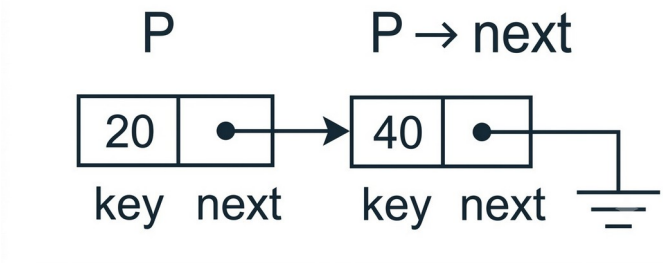
One more structure is created and p→next_ points to it. Key field in this new

structure is assigned 40.

Check the diagram.

p->next_ = (node_t*)malloc(sizeof(node_t));

p->next_->key_ = 40;



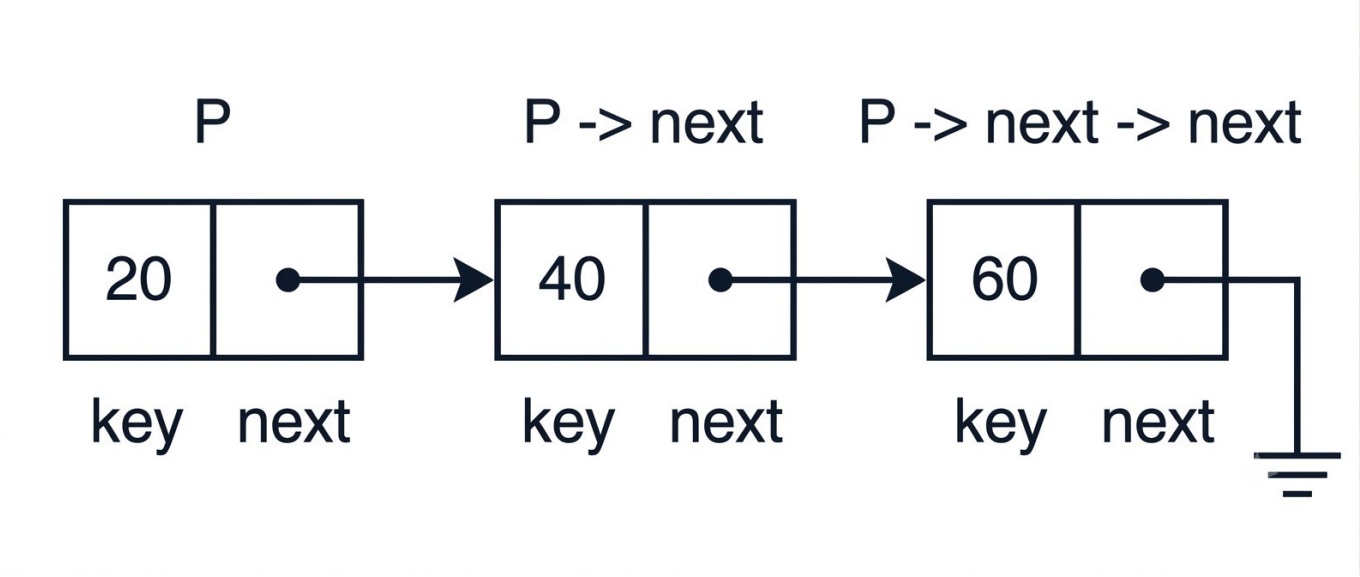
One more structure is created. Link field of the earlier structure points to this

Link field of the new structure is grounded (assigned NULL). So, the list ends.

```
p->next_->next_ = (node_t*)malloc(sizeof(node_t));

p->next_->next_->key_ = 60;

p->next_->next_->next_ = NULL;
```



```
// 2_ex.c

#include <stdio.h>

#include <stdlib.h>

struct node
{
        int key_;

        struct node *next_;
};
typedef struct node node_t;

void disp(node_t* q)
{
        while(q != NULL)
        {
                printf("%d ", q->key_);

                q = q->next_;

        }
}



void freelist(node_t* q)
{
        node_t* r;

        while(q != NULL)

        {

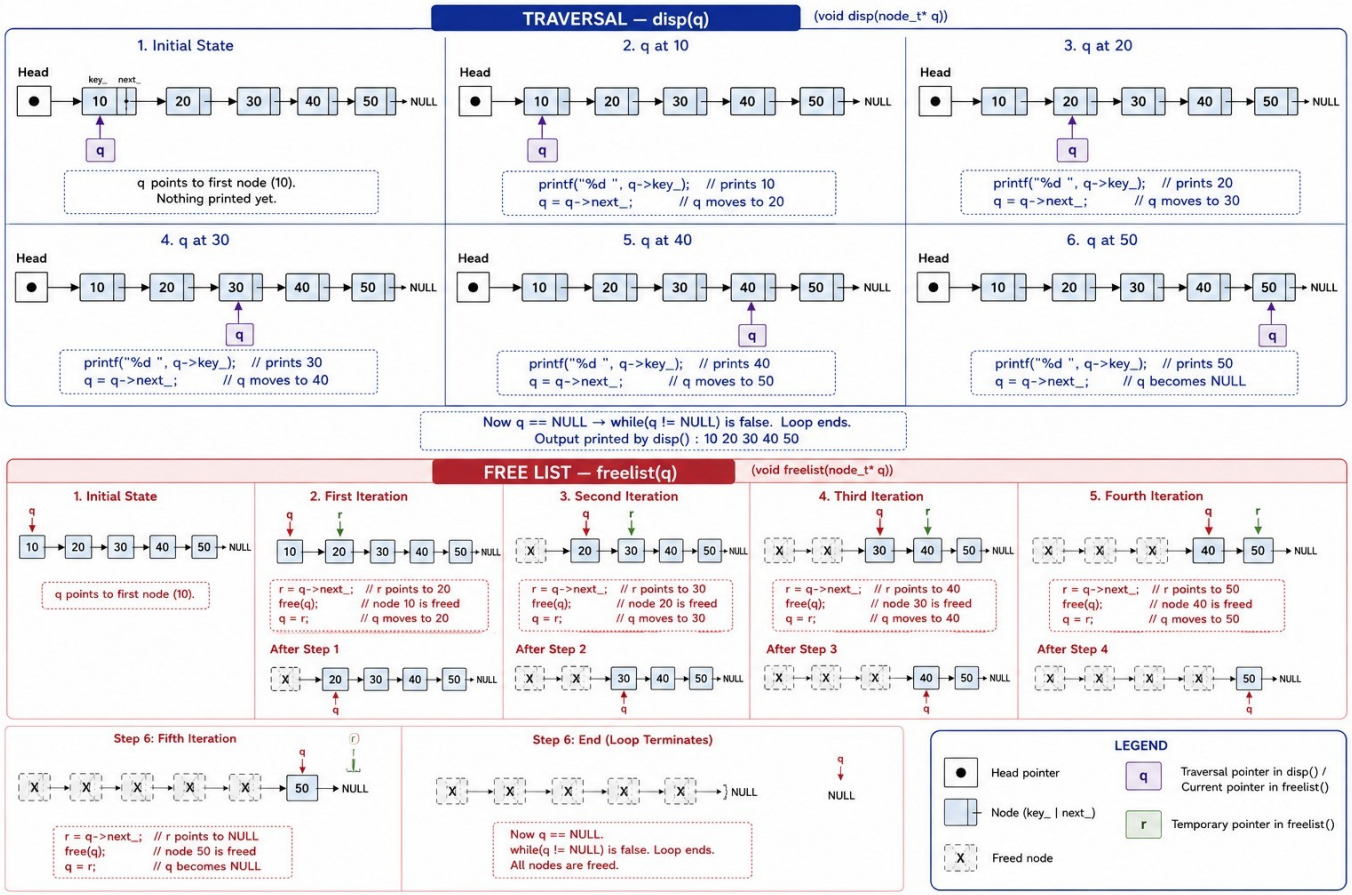
                r = q->next_;

                free(q);

                q = r;

        }

}
```


TRAVERSAL — disp(q)
(void disp(node_t* q))
1. Initial State
Head
key_ next_
10 20 30 40 50 NULL
q
q points to first node (10).
Nothing printed yet.
2. q at 10
printf("%d ", q->key_); // prints 10
q = q->next_; // q moves to 20
3. q at 20
printf("%d ", q->key_); // prints 20
q = q->next_; // q moves to 30
4. q at 30
printf("%d ", q->key_); // prints 30
q = q->next_; // q moves to 40
5. q at 40
printf("%d ", q->key_); // prints 40
q = q->next_; // q moves to 50
6. q at 50
printf("%d ", q->key_); // prints 50
q = q->next_; // q becomes NULL
Now q == NULL → while(q != NULL) is false. Loop ends.
Output printed by disp() : 10 20 30 40 50
FREE LIST — freelist(q)
(void freelist(node_t* q))
1. Initial State
q points to first node (10).
2. First Iteration
r = q->next_; // r points to 20
free(q); // node 10 is freed
q = r; // q moves to 20
After Step 1
3. Second Iteration
r = q->next_; // r points to 30
free(q); // node 20 is freed
q = r; // q moves to 30
After Step 2
4. Third Iteration
r = q->next_; // r points to 40
free(q); // node 30 is freed
q = r; // q moves to 40
After Step 3
5. Fourth Iteration
r = q->next_; // r points to 50
free(q); // node 40 is freed
q = r; // q moves to 50
After Step 4
Step 6: Fifth Iteration
r = q->next_; // r points to NULL
free(q); // node 50 is freed
q = r; // q becomes NULL
Step 6: End (Loop Terminates)
Now q == NULL.
while(q != NULL) is false. Loop ends.
All nodes are freed.
LEGEND
Head pointer
Node (key_ | next_)
Freed node
Traversal pointer in disp() / Current pointer in freelist()
Temporary pointer in freelist()

```
int main()

{

        node_t *p;

        p = (node_t*)malloc(sizeof(node_t));

        p->key_ = 20;

        p->next_ = (node_t*)malloc(sizeof(node_t));

        p->next_->key_ = 40;



        p->next_->next_ = (node_t*)malloc(sizeof(node_t));

        p->next_->next_->key_ = 60;

        p->next_->next_->next_ = NULL;

        disp(p);

        freelist(p);

}
```

p points to the beginning of the list - points to the frst node of the list. p is referred as the head of the list. Given the head of the list, we can traverse or walk through the list. Let us examine the disp function. The parameter q gets a copy of p. q points to the frst node in the list.

printf("%d ", q→key_);

q→key_ displays the key field in the frst node.

We should move to the next node. We cannot do q++ as the nodes are not necessarily allocated contiguous memory unlike arrays. The link field in the node to which q points to, points to the next

node. So, we copy the link field of q to q. The statement below is the most important statement in data

structures.

q = q→next_;

we repeat this until q becomes NULL signifying that we have come to the end of the list.

The function freelist releases the memory occupied by the linked list.

Observe that we should not say

free(p);

This would release the frst node and rest of the list becomes garbage.

```
r = q->next_;
free(q);
q = r;
```

Observe that before removing what q points to, we should copy the link field so that we can change q to point to the next node after deleting what q pointed to before. The order of statements is very important. This technique of storing a pointer value temporarily before restoring it into some variable is very commonly used in data structures.

Ordered list:

// client.c

An ordered list has nodes arranged in the order of key based on some predicate. Let us try to create a list of nodes in the non-decreasing order of keys.

The list structure is opaque for the user. The user should not directly access anything within this structure as this could change.

Our implementation of list structure has the head of the list within it. Our implementation uses self referential structure node_t.

The client code initializes the list. The empty list normally grounds the head – makes it NULL.

The client code inserts elements into the list by calling the function insert_list.

The insert_list function creates a node, fills the key field, grounds the link field and then inserts the node in the right position in the list.

There are four cases to consider.

a) empty list

b) insert in the middle

c) insert at the end

d) insert in the beginning

```
if(ptr_list->head == NULL)
{
        ptr_list->head = temp;
        temp->next_ = NULL;
}
```

The code above takes care of the empty list case. The head being NULL indicates that the list is empty. Change the head to point to the new node created and ground the pointer of the new node.

To insert in a non-empty list, we have to find the position to insert. We make a pointer pres point to the first node of the list. Compare the key in node pointed to by pres to the key in new node pointed to by temp. if the former is less than the latter, we move to the next node by changing pres. pres = pres→next_. We realize that we should have inserted only after we have crossed the point. **It is like traveling in one way traffic**. We cannot move back.

We can overcome this problem by having a prev pointer tail before the pres as

shown in the code. The initial value of prev is NULL.

```
node_t* prev = NULL;

node_t* pres = ptr_list->head;

while(pres != NULL && pres->key_ < temp->key_ )

{

        prev = pres;

        pres = pres->next_;

}
```

We exit the loop in two ways. Either we have found a position to insert or the we have reached the end of the list.

In case of insertion in the beginning of the list, head has to be changed. We make out that the insertion has to be in the beginning, if the prev is still NULL after the loop is exited.

```
if(prev == NULL) // insert in the beginning

{

        temp->next_ = pres;

        ptr_list->head = temp;

}

else // middle or end

{

        temp->next_ = pres;

        prev->next_ = temp;

}
```

We also note that the code for insertion at the end or in the middle is same.

We insert between the nodes pointed to by prev and pres. Next_ of prev points to what temp points to and link of temp points to what pres points to.

The rest of the functions are self explanatory.

```
#include <stdio.h>

#include "mylist.h"

int main()

{

        mylist_t l;

        init_list(&l);

        int a[] = { 20, 10, 50, 30, 40};

        int n = 5;

        for(int i = 0; i < n; ++i)

        {

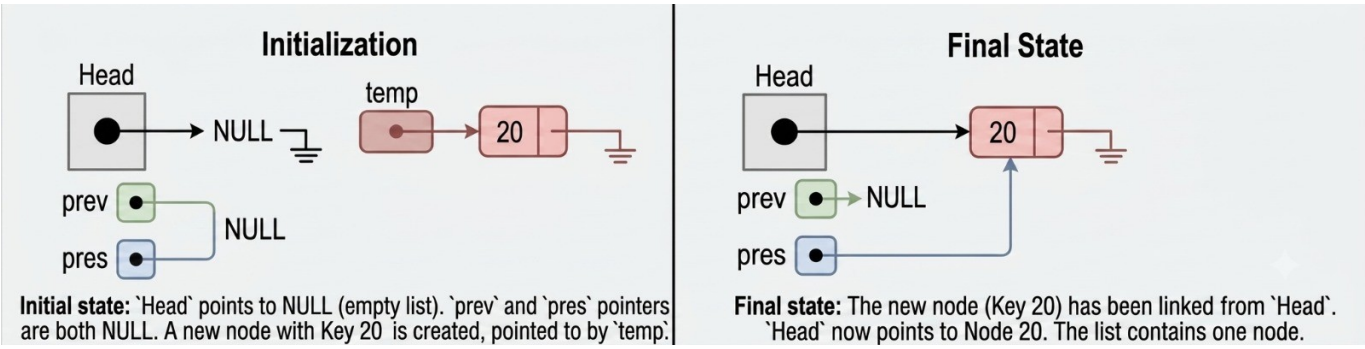
                insert_list(&l, a[i]);

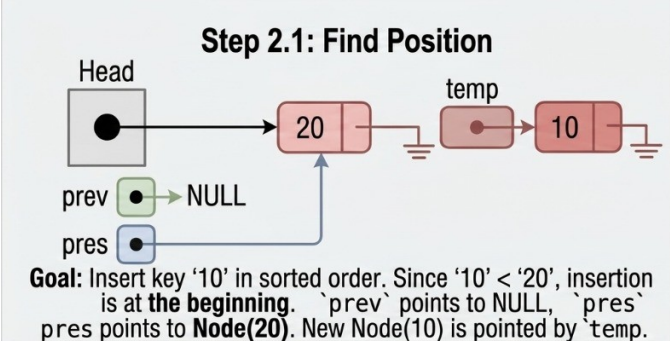
        }

                disp_list(&l);

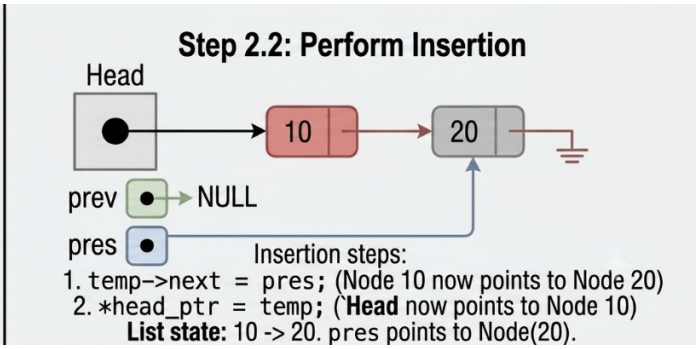
                free_list(&l);

}
```



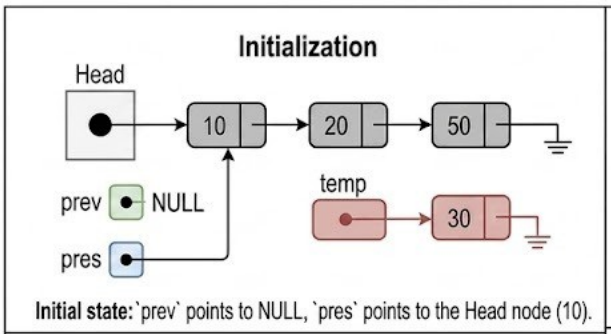


Goal: Insert key '10' in sorted order. Since '10' < '20', insertion is at **the beginning**. `prev` points to NULL, `pres` `pres` points to **Node(20)**. New Node(10) is pointed by `temp`.

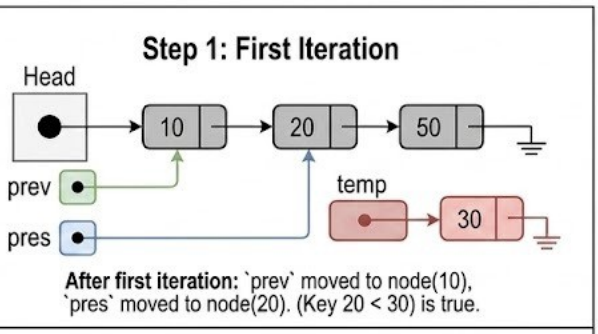


Insertion steps:
1. `temp->next = pres;` (Node 10 now points to Node 20)
2. `*head_ptr = temp;` (`Head` now points to Node 10)

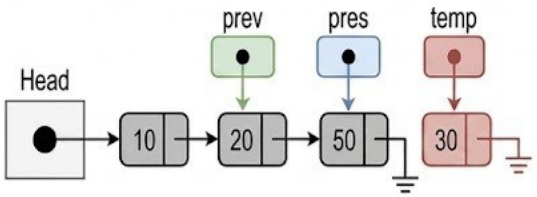
List state: 10 -> 20. `pres` points to Node(20).



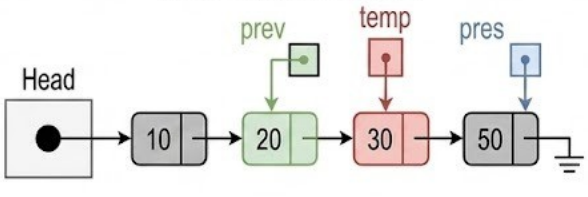
Initial state: `prev` points to NULL, `pres` points to the Head node (10).



After first iteration: `prev` moved to node(10), `pres` moved to node(20). (Key 20 < 30) is true.



Iteration 2: `prev points to node(20), pres moved to node(50). Loop condition check: (Key 50 < 30) is false. Looo condition fails.



Loop Exits.
Final Pointers `prev` and `pres` confirm position (between 20 and 50).
Inserted node(30) at position.

```
//  mylist.h

#ifndef MYLIST_H

#defne MYLIST_H

struct node

{

        int key_;

        struct node *next_;

};

typedef struct node node_t;

struct mylist

{

        node_t *head;

};

typedef struct mylist mylist_t;

void init_list(mylist_t*);

void insert_list(mylist_t*, int key_);

void disp_list(mylist_t*);

void free_list(mylist_t*);

#endif
```

```
// mylist.c

#include <stdio.h>

#include "mylist.h"

#include <stdlib.h>

void init_list(mylist_t* ptr_list)

{

	ptr_list->head = NULL;

}

// 1. empty list

// 2. insert in the middle

// 3. insert in the beginning

// 4. insert at the end

void insert_list(mylist_t* ptr_list, int key)

{

	// create the node

	node_t* temp = (node_t*)malloc(sizeof(node_t));
```

```
temp->key_ = key; temp->next_ = NULL;

// 1.empty list

if(ptr_list->head == NULL)

{

        ptr_list->head = temp;

        temp->next_ = NULL;

}

else

{

        // traverse

        node_t* prev = NULL;

        node_t* pres = ptr_list->head;

        while(pres != NULL && pres->key_ < temp->key_ )

                {

                prev = pres;

                pres = pres->next_;

                }

        if(prev == NULL) // insert in the beginning

        {

                temp->next_ = pres;

                ptr_list->head = temp;

        }
```

```
                else // middle or end
                {
                        temp->next_ = pres;
                        prev->next_ = temp;
                }
        }
}
```

```
void disp_(node_t* q)
{
	while(q != NULL)
	{
		printf("%d ", q->key_);
		q = q->next_;
	}
}
void disp_list(mylist_t* ptr_list)
{
	disp_(ptr_list->head);
}
void free_list_(node_t* q)
{
	node_t* r;
	while(q != NULL)
	{
		r = q->next_;
		free(q);
		q = r;
	}
}
```

```
void free_list(mylist_t* ptr_list)
{
        free_list_(ptr_list->head);
}
```

We have been successful in separating the interface from the implementation. The client can use it independently without worrying about the changes to the implementation.

How do we capture the dependency between files and the commands to be executed when the dependent files change? The tool used for this purpose is called a build tool. **Make** build automation tool was designed and created by **Stuart Feldman** at Bell Labs in **April 1976**. In mingw, there is a similar tool called mingw32-make. In Ubuntu and Mac, the tool is called make.

Create a file – called makefile by default.

Indicate the binary – output – file then : followed by names of all files on which this target depends.

In the next line, start with a tab and indicate the commands to be executed to make the target.

Repeat this for every target file.

Makefile :

```
a.exe : mylist.o client.o
        gcc mylist.o client.o
mylist.o : mylist.c mylist.h
```

```
	gcc -Wall -c mylist.c

client.o : client.c mylist.h

	gcc -Wall -c client.c
```

To execute the makefile, say

Mingw32-make

The makefile holds dependency information and executes only those commands to remake the target when the files on which the target depends change.

So surprisingly, this tool has not changed for over 55 years!

Linked List – continued

So far, we have said that the list is a data structure which has elements identifiable by ordinal positions. It has two properties.

1. Access to an element is sequential – no direct access.
2. Insertion and deletion once the position is known, requires a few operations. It does not depend on the number of elements in the list.

We have seen that the list can be implemented as a linked list.

We have also created an ordered list where the nodes in the list are arranged based on a key field. We have also discovered that the insert operation has 4 different boundary conditions.

Can we change the implementation without affecting the client? Why not? A data structure is an ADT – abstract data type. It specifies the interface through the header file in 'C'. If we change the implementation file without changing the header file, then the client code need not be even recompiled.
Note: In certain cases, in C and C++, the client code need to be recompiled without changing the code.

We will compile our client code and keep it separate. Please note the time stamp by using the dir or ls -l command.

We will change the concept of our list. We will say that our empty list will have one node. So, while initializing the list, we will create one dummy node – called the header node. In our example, we may not initialize the key_ field. Of course, we will ground the next_ field.

```
void init_list(mylist_t* ptr_list)
{

        ptr_list->head_ = (node_t*)malloc(sizeof(node_t));
        ptr_list->head_->next_ = NULL;
}
```

Since the list has a dummy header node, we may have to change all other functions.

The display function should skip one node and then start displaying.

```
void disp_list(mylist_t* ptr_list)

{

        disp_(ptr_list->head_->next_);
}
```

The function free list goes back to the beginning, ready to build the list again. The list has one dummy node with its next_ field grounded.

What is left? Insert function. Let us examine the function. We said earlier that there are four boundary conditions which should take care.

1. Empty list : where the head_ which is NULL has to be changed. But in our case, head_ is never NULL. So, this clause is not even necessary.
2. Insert in the beginning of the list requiring head_ to be changed. This also will not happen.
3. & 4. Insert in the middle or the end. Let us examine as we go by.

Let us examine the code for insertion.

```
void insert_list(mylist_t* ptr_list, int key)
{
        // create the node
        node_t* temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key; temp->next_ = NULL;

        // This goes!!
        // 1.empty list
        if(ptr_list->head_ == NULL)
        {
                ptr_list->head_ = temp;
                temp->next_ = NULL;
        }
```

```
        Else // NO else
        {
                // traverse // requires changes in initialization
                node_t* prev = NULL;
                node_t* pres = ptr_list->head_;

                while(pres != NULL && pres->key_ < temp->key_ )
                {
                        prev = pres;
                        pres = pres->next_;
                }
                // this also goes
                if(prev == NULL) // insert in the beginning
                {
                        temp->next_ = pres;
                        ptr_list->head_ = temp;
                }
                else // middle or end
                {
                        temp->next_ = pres;
                        prev->next_ = temp;
                }

        }


}
```

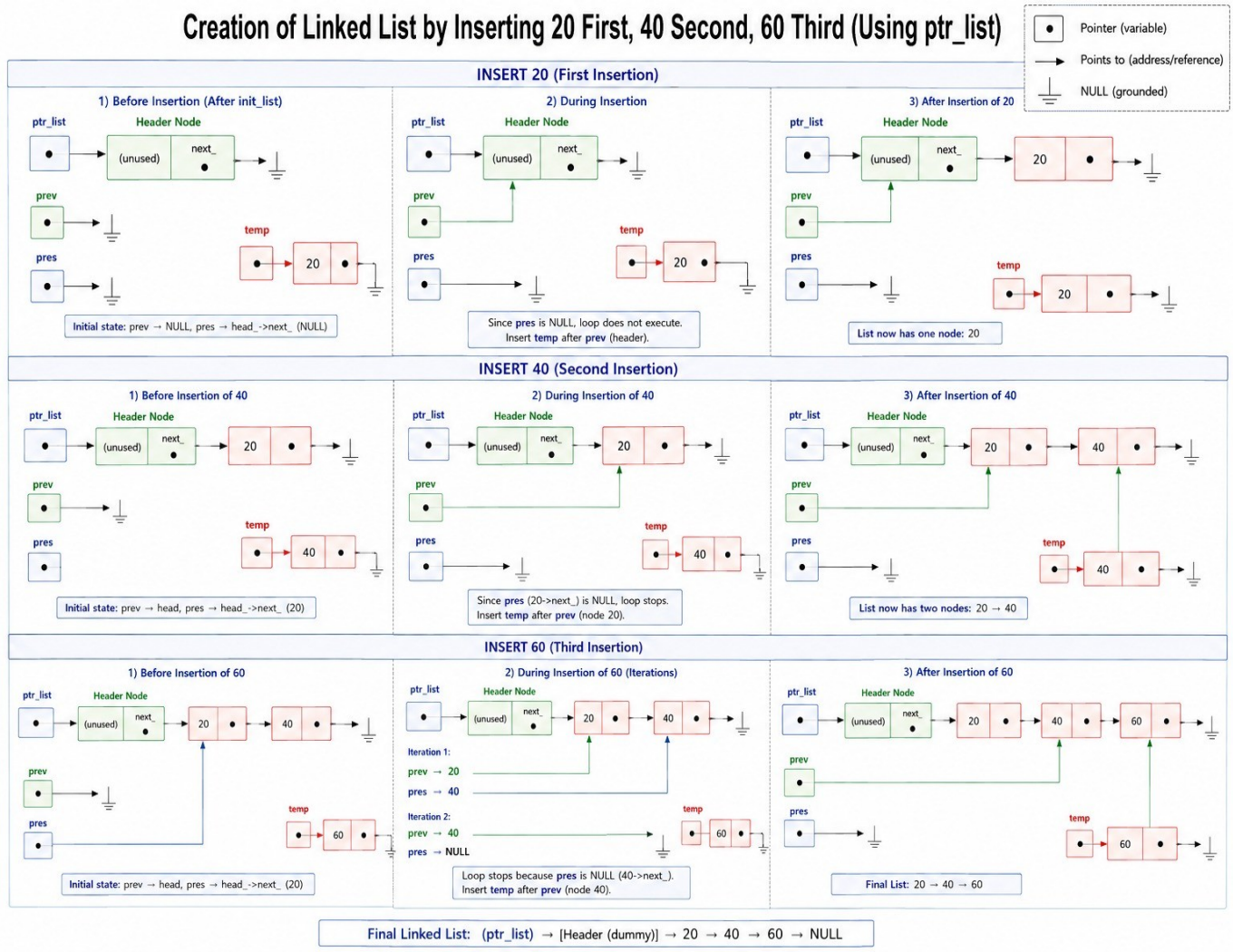
Now the changed function looks like this.

```
// modified code
void insert_list(mylist_t* ptr_list, int key)
{
        // create the node
        node_t* temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key; temp->next_ = NULL;

        // traverse
        // observe the changes here
        node_t* prev = ptr_list->head_;
        node_t* pres = ptr_list->head_->next_;


        while(pres != NULL && pres->key_ < temp->key_ )
        {
                prev = pres;
                pres = pres->next_;
        }

        temp->next_ = pres;
        prev->next_ = temp;
}
```



This looks so simple. Is this not easy to read? Our insertion function has become extremely readable.

So, we use the header node to make insertion simple.

The client is not affected. We can recompile this new code, link with the object file of the client and run the code.

Similarly can we have one more node at the end – tail or tailer node? If yes, what could be the reason for it?

// empty list : diagram
// insert into this list : no change in head : diagram

```
// insert in the middle
// insert at the beginning
// insert at the end
```

Stack

Concept of ADT : Revision :

So far, we have discussed that a type is an interface – it states what we can and does not tell us how it does it. We have also said that a language cannot provide all possible types. Some languages provide a mechanism to make out own type. In others, we can simulate it.

We have also said that there are different ways of arranging information. Each is geared for a particular purpose. These are called data structures. A data structure is an abstract data type. It tells the user what operations it supports. It does not commit to any particular implementation. A client or the user has the flexibility of selecting from various vendors supporting that interface. A developer can support several clients.

We have also realized that the interface is sacred and that it should not be changed. We can change the implementation without telling the user. In languages like 'C' and C++, the client may have to recompile the code in the worst case.

Stack : ADT :

- Is a data structure
- Has zero or more components
- Can operate at only one end – normally called the stack top
- Supports LIFO : Last In First Out
- What goes in last, comes out first
- Does not support the concept of traversal

- **Interface** :
    - Create
    - Destroy
    - Push : add an element at the top
    - Pop : remove an element at the top
    - Peek : check the top element without removing it
    - Is_empty : check whether the stack is empty
    - Is_full : check whether the stack is full

- **Implement** using
    - A linked list
    - An Array
-

Applications of stack :

Let us discuss a few applications of the stack.

1. **Stack frame or activation record** : When a function is called, an activation or a stack frame is created. The parameters, the local variables, the temporary variables the compiler creates and the return address are stored in this piece of memory. If the called function calls another, another activation record is created on top of this. When the function returns, the activation record is removed.
2. **Think : Would every stack frame have the same size? How to we remove it?**

3. **Redo-undo operations** : Do you remember pressing <ctrl>z in most of the editors? To support such an operation, the system should track the operations applied. It should undo the last operation first. So, it requires a stack.

4. **Parentheses matching**: To overcome the issue with the fixed precedence and association of operators and to change the order of evaluation, we use parentheses. The compiler will have to check whether the parentheses are well formed. Not only the number of left parentheses should match the number of right parentheses, but the left one should appear

before the right. We use a stack to check whether the right parenthesis in the expression matches the left one on the top of the stack.

You may remember that we studied 4 types of brackets in our high school days.

1. Vinculum
2. Simple brackets or parentheses
3. Curly or flower brackets
4. Square or big brackets.
5.

If a language were support all but the first – first cannot be supported directly – can we use the stack to check whether such an expression is well formed?

5. **Expression Evaluation**: We normally write expressions with operators in between the operands. This is called infix expression. The evaluation of operators depends on both the precedence and association of operators. Would it be possible to create an expression where all the operators are at the same level of precedence and have left to right association? In such a case, we can evaluate the expression as soon as the operator is reached. Such an expression is postfix expression where the operator appears after the operator. We can convert an infix expression to a postfix expression by using a stack of operators and we can evaluate an expression by using a stack of operands.
   We will discuss this at length later in this course.

   There are lots of applications using stack. We will mention them. You will come across them later in your study.

   1. Depth first search of a graph
   2. Backtracking
   3. Branch and bound
   4. Simulation of recursive functions

      …

Some interesting diagrams for you to view:

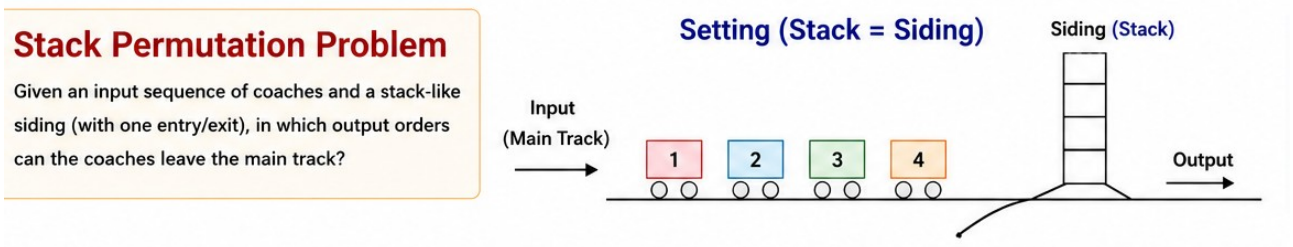


I am sure you would have seen this in restaurants. You can pull the top place or push a plate at the top.

An interesting problem for you to solve. There are a few coaches on the track. The order can be changed by using a sliding stack. There are three possible actions.

1. Coach goes directly to the output track.
2. Coach is pushed into the sliding stack
3. Coach is popped out from the sliding stack.

At the end, all coaches are popped out.



Stack Implementation

Let us implement the stack in two ways.

1. Stack using an array
2. Stack using a list

Array implementation of the stack:

Interface:

Let us capture the interface in the stack.h file.
We will create an ADT called stack_t. We will support the following functions on this type.

Please refer to stack.h in the folder stack_using_array.

This will initialize the stack.
void init_stack(stack_t *ptr_stack);

This will remove all the elements from the stack
void deinit_stack(stack_t *ptr_stack);

We are implementing a stack of character. We could of course choose any other valid type of component. The function push will add a character at the top of the stack.
void push(stack_t *ptr_stack, char ch);

The function pop removes the top character from the stack and returns it.
char pop(stack_t *ptr_stack);

We cannot pop if the stack is empty. We can check the condition using the function is_empty.
int is_empty(stack_t *ptr_stack);

We cannot push if the stack is full. We can check the condtion using the function is_full.
int is_full(stack_t *ptr_stack);

In the last two exceptional cases, the way of handling depends on the user. We are not discussing this at present.

Implementation:

We will implement the stack as having an array of the component type. Unfortunately, this has to be part of the header file. This forces the client to recompile his code if this is changed. But he does not have to modify his code.

The interface would remain same even if we implement the stack using a linked list but the structure list_t.

Let us look at the stack.h file.

```
#define MAXSIZE 5
struct stack
{
        char key_[MAXSIZE];
        int top_;
};
typedef struct stack stack_t;
```

Observe that apart from the array of component type – in our case character, we have another field top_. This holds the index of the top element.

Either this integer field could be the # of elements in the stack or the index of the top element. Based on this all functions will change. We shall track the index of the top element of the stack using the variable top_.

The functions are implemented in stack.c file.

Initialize stack :

void init_stack(stack_t *ptr_stack);

This function initializes top_ to -1. This is an invalid index on the array. Every variable we create should be associated with some meaning. In this case, top_ is the index of the last element added and is not the index of the position where the next element can be added. This decision affects all the functions we implement.

Client Program:

We have written a simple program to test this stack implementation. The file is client.c

We create and initialize a stack of characters. We read a string from the keyboard. We traverse the string and push each character on to the stack. Then we pop the characters and display until the stack becomes empty.

So, we will get the input string in the reverse order.

We shall discuss the list implementation in the next.

Deinitialize stack:

void deinit_stack(stack_t *ptr_stack);

This will reinitialize the top_ to -1 indicating that the stack has conceptually become empty.

Push a character into the stack:

void push(stack_t *ptr_stack, char ch);

This will increment the top_ and then check whether there is space by calling is_full. If there is no space, it gives a message. Otherwise, the character is placed on the top of the stack.

Pop a character from the stack:

char pop(stack_t *ptr_stack);

This function checks whether the stack is not empty. If empty, this function gives a message, otherwise, takes the top element out, decrements the top_ and returns the element taken out.

int is_empty(stack_t *ptr_stack);

This checks whether the stack is empty by comparing top_ with -1. This function should be used by pop before using top_.

int is_full(stack_t *ptr_stack);

As we are implementing the stack using an array, it is possible that at some time the array becomes full. This function compares the value of top_ and the array size to determine whether the stack is full. This function should be used before pushing into the stack.

List implementation of the stack:

Interface:

All the function declarations remain the same. We do not want the client to modify his code. At the most he may have to recompile the code.

So, the functions are :

```
void init_stack(stack_t *ptr_stack);
void deinit_stack(stack_t *ptr_stack);
void push(stack_t *ptr_stack, char ch);
char pop(stack_t *ptr_stack);
int is_empty(stack_t *ptr_stack);
int is_full(stack_t *ptr_stack);
```

We will change the structure of stack to have a pointer to a node where the node has a character key_ and a pointer to a node next_ - node is a self referential structure. This change requires that the client needs to recompile the code.

```
struct node
{
        char key_;
        struct node* next_;
};
typedef struct node node_t;

struct stack
{
        node_t* top_;
};
typedef struct stack stack_t;
```

implementation of stack using the list:

Most of these functions are fairly straightforward.

Init_stack :

This will initialize top_ to become NULL indicating that the stack is empty.

void init_stack(stack_t *ptr_stack);

deinit_stack :

This has not been implemented. This requires that we pop nodes repeatedly until the stack becomes empty.

void deinit_stack(stack_t *ptr_stack);

push:

This function will check whether the stack is not yet full. If so, creates a new node, populates it and then adds to the stack. The new node created will have its next_ field point to wherever top_ points to and then top_ is changed to point to the new node created.

You may observe that the order of these operations matters.

void push(stack_t *ptr_stack, char ch);

pop:

This function whether the stack is empty. If not, it returns the key_ in the top element and also removes it. There are a few steps. **The order of steps matters**.

- Copy the key_ to a local variable
- Create a pointer to point to the top element – make the variable have the value of top_
- Change top_ to point the next node in the stack
- Free the node to which the local pointer points to
- Return the key_ copied to a local variable

char pop(stack_t *ptr_stack);

is_empty:

This function checks whether top_ is NULL.

int is_empty(stack_t *ptr_stack);

is_full:

This function is tricky. How to make whether the heap has memory to allocate? There is no simple way. We will try allocating calling malloc. If it fails, is_fill returns true – that is 1. If it succeeds, this function returns false – that is 0. But we should free the memory allocated. Otherwise garbage is created leading to memory leak.

```
int is_full(stack_t *ptr_stack);
```

So, we have a type called stack implemented in two ways. The client can select whichever he wants. Is this not so wonderful?

Applications using Stack

Stack :

We have discussed the **Abstract Data Type** stack. We have said that it is a data structure where all operations happen at one end – called the stack top. We can push an element to the stack, pop an element, peek the top of the stack, check for stack being empty and check for stack being full. Nothing else is supported by this type.

There are many applications which require this data structure. In formal languages, we talk about a set of languages which require such a data structure – called push down automaton. Almost all programming language features belong to this category. These languages are called context free languages. You will study sometime later in your curriculum.

Well Formed Parentheses:

Let us discuss one example in this document. How to find whether a given expression has well performed parentheses? For example,

(3 + 4)

((5 + 6) * (7 + 8))

These are well performed. For making our life easier, we can ignore all other characters other than parentheses (and).

So, the above expression can be written as

()

(()())

Clearly, these are not well formed.

(()

())

So, one of the requirements is that the number of left parentheses should be equal to the number of right parentheses. Is that sufficient? If so, we may not require a stack at all.

What if the parentheses are like this?

)(

())(())

These are not well formed. A right parenthesis occurs before the left one!

So, what is the algorithm we shall follow to check the well formedness of the parentheses?

- Initialize the stack

  ```
  stack_t mystack;
  init_stack(&mystack);
  ```

- Read the given expression. This is basically a string.

  ```
  char str[MAXSIZE + 1];
  scanf("%s", str);
  ```

- Walk through the string character by character
  - If the character is a left parenthesis, put a marker that we have seen a left parenthesis and that we can expect a corresponding right parenthesis.

    Push a place holder onto the stack
  - If the character is a right parenthesis, pop the place holder from the stack

    If the stack is empty, then we have got a right parenthesis before the left. The expression is not well formed. Exit the loop and give the message that the expression is not well formed.

```
int is_valid = 1;
char ch; char *s = str;
while( is_valid && (ch = *s++) != '\0')
{
        if(ch == '(')
        {
                push(&mystack, 'x'); // some placeholder
        }
        else if(ch == ')')
        {
                if(! is_empty(&mystack))
                        pop(&mystack);
                else
                        is_valid = 0;
        }
}
```

- At this point, the stack should be empty indicating that for every left parenthesis encountered, we have also encountered the corresponding right parenthesis in order.
- If the stack is empty, give the message that the expression is well formed else give the message that the expression is not well formed.

```
if(is_valid && is_empty(&mystack))
{
        printf("Well formed expression\n");
}
else
{
        printf("not well formed expression\n");
}
```

Please observe that we use the stack like any other type we know. We create a variable of type stack like the way we would have created an int variable. We initial the stack. We then perform the

valid operations on the stack the way we would perform addition, subtraction on int. We do not know how addition of int is carried out in the computer and we do not worry about it. Similarly we do not have to know how the operations on the stack are performed. We are not required to know them to use the stack.

To use a type, we should know the interface and we do not have know the implementation.

Well formed brackets:

In school days, we have learnt 4 types of brackets – vinculum, simple brackets or parentheses, flower brackets or curly brackets, big or square brackets. We cannot type vinculum. What if an expression contains all these types of brackets? How do we check whether such an expression is well formed?

For example,

(){}[]

([]{})

([{}])

are all well formed.

([)]

([]])

(][)

are not well formed.

We can do that by using the stack.

In that example whenever we encountered a left parenthesis, we pushed a place holder on the stack, and we removed that whenever we encountered a right parenthesis.

This is not enough when we have multiple types of brackets. If we encounter a right], we should have encountered a left [before this.

So, for every type of left bracket we encounter, we should put a different type of marker on the stack. When we get a right bracket, we should be able to pop and match whether the marker for the corresponding left bracket was on the stack.

```
char str[MAXSIZE + 1];
char *s = str;
int is_valid = 1;
char ch;
while( is_valid && (ch = *s++) != '\0')
{
        if(ch == '('
        {
```

```
                push(&mystack, ')');
        }
        else if(ch == '[')
        {
                push(&mystack, ']');
        }
        else if(ch == '{')
        {
                push(&mystack, '}');
        }
        else if(ch == ')' || ch == ']' || ch == '}')
        {
                if(! is_empty(&mystack))
                {
                        if(ch != pop(&mystack))
                        {
                                is_valid = 0;
                        }
                }
                else
                        is_valid = 0;
        }
}
```

Let us observe the change in the code compared to the previous one. As we should distinguish the type of the bracket we require a different type of placeholder for each. We may as well choose the corresponding right bracket to push into the stack.

When we encounter a right bracket, the stack should not be empty. If it is empty, the expression is not valid. The top character should match the right bracket in the stack. Otherwise the expression is not valid.

Once the expression has been complete processed, the stack should be empty for the expression to be valid.

```
if(is_valid && is_empty(&mystack))
{
        printf("Well formed expression\n");
}
else
{
        printf("not well formed expression\n");
}
```

Observe that we have used the stack without knowing whether the stack is implemented using an array or a list. Are you not thrilled?

Applications using Stack

Evaluation of Expression :

It is so complicated to evaluate an expression. We had something called BODMAS rule.

- **B** — Brackets (solve things inside ( ), [ ], { } first)
- **O** — Orders (powers, square roots, or indices)
- **D** — Division
- **M** — Multiplication
- **A** — Addition
- **S** — Subtraction.

It was very simple. But in programming languages like 'C', we have multiple levels of precedence. So also we have variation of associative rules. Nesting of parentheses adds one more level of complication.

Can we have expressions which can be evaluated as soon as the operator is encountered?

In the normal way of writing, we write the operator between the operands. These are some simple examples.

3 + 4

4 * 5 + 6

4 + 5 * 6

5 * 6 + 7 * 8

These are called **infix expressions** – the operators are between the operands.

Can we have expressions where the operator appears after the operands?

Let us relook at the expressions above.

3 4 +

4 5 * 6 +

4 5 6 * +

5 6 * 7 8 * +

These are called **postfix expressions** – the operators are after the operands.

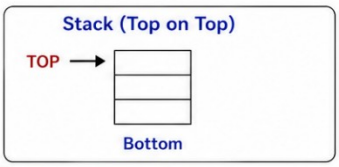
In postfix expressions, all operators at the same level of precedence. All operators are left associative. We can create a stack of operands. As soon as an operator is encountered, we pop the operands from the stack, evaluate the expression based on the operator, push the result back into the stack. When the expression is completely processed, the stack should have a single value which is the result.

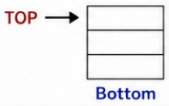
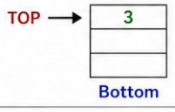
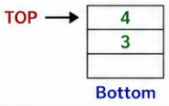
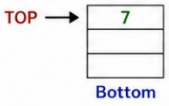
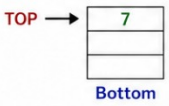
We will discuss how to convert an infix expression to postfix expression later in this document.
We will try to understand how stack is used to evaluate each of the expressions above.
These diagrams are generated using genAI tool.

Example 1 : 3 4 +

Evaluate Postfix Expression: 3 4 +

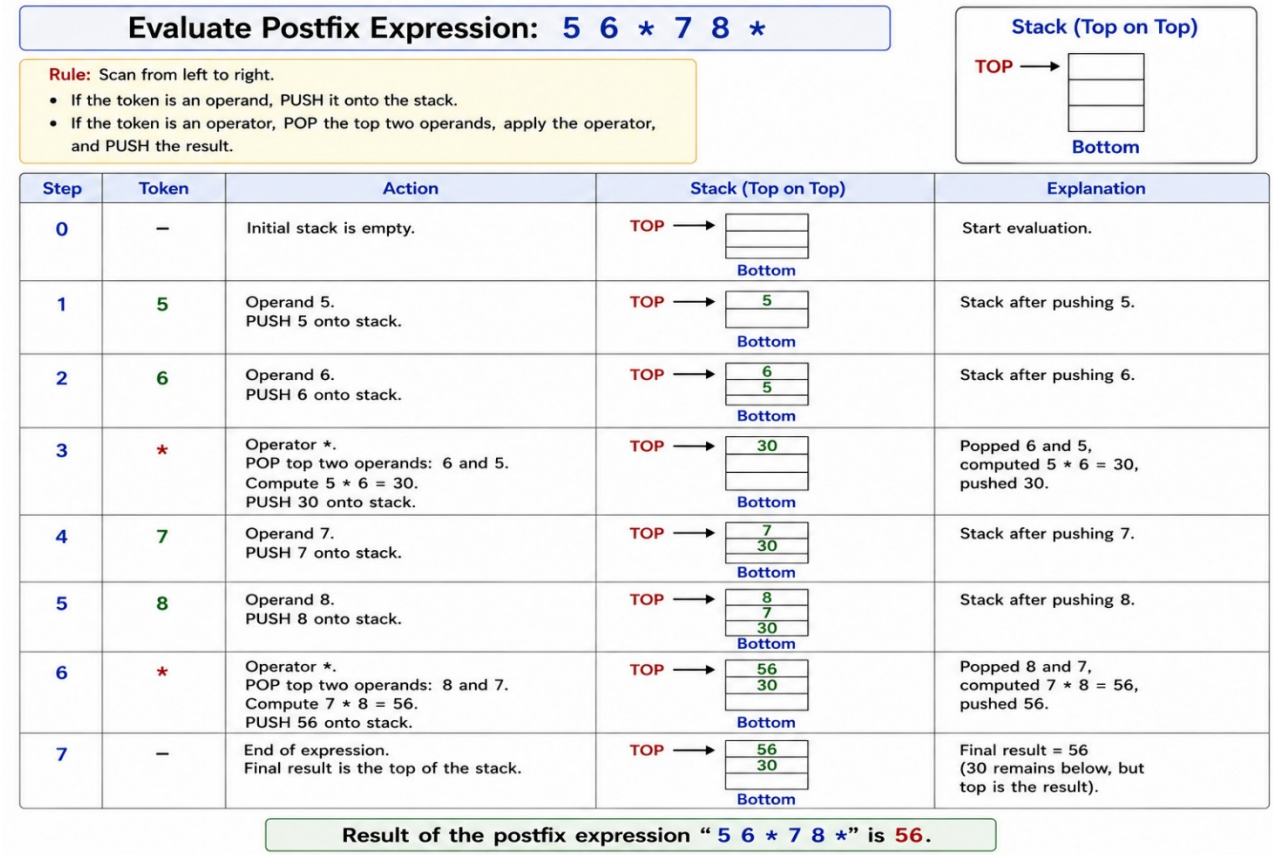
Rule: Scan from left to right.
If the token is an operand, PUSH it onto the stack.
If the token is an operator, POP the top two operands, apply the operator, and PUSH the result.



Step	Token	Action	Stack (Top on Top)	Explanation
0	–	Initial stack is empty.		Start evaluation.
1	3	Operand 3. PUSH 3 onto stack.		Stack after pushing 3.
2	4	Operand 4. PUSH 4 onto stack.		Stack after pushing 4.
3	+	Operator +. POP top two operands: 4 and 3. Compute 3 + 4 = 7. PUSH result 7 onto stack.		Popped 4 and 3, computed 3 + 4 = 7, pushed 7.
4	–	End of expression. Final result is the top of the stack.		Final result = 7

Result of the postfix expression "3 4 +" is 7.

Example 2 : 5 6 * 7 8 * +



NOTE : This image has an error. No correction is made to show that these genAI tools do give wrong results. We cannot always trust them. They hallucinate!

Algorithm for evaluation of postfix expression :

We shall assume that between the operands and operators there is white space. This makes our life easier.

- Read the postfix expression

  ```
  char str[MAXSIZE + 1];
  gets(str);
  printf("input expression : :%s:\n", s);
  ```

- Create a stack of operands. We are assuming that we will support only integer arithmetic. The component of the stack is made int. Initialize the stack.

  ```
  stack_t mystack;
  init_stack(&mystack);
  ```

- Walk through the given postfix expression

  There are a few initializations. One of them is val. This is used to collect multiple digits, multiply by 10 and add to form the numeric value of the number formed by these digits.

  ```
  char *s = str;
  char ch;
  int val1; int val2;
  int val = 0;
  int is_num = 0;
  ```

  - If the character is an operator, pop as many operands as required, do the operation and push the result back into the stack.

  ```
  while( (ch = *s++) != '\0')
  {
          if(ch == '+' || ch == '-' || ch == '*' || ch == '/')
          {
                  val1 = pop(&mystack);
                  val2 = pop(&mystack);
                  switch(ch)
                  {
                          case '+' : push(&mystack,val2 + val1);
  ```

```
                    break;
            case '-' : push(&mystack,val2 - val1);
                    break;
            case '*' : push(&mystack,val2 * val1);
                    break;
            case '/' : push(&mystack,val2 / val1);
                    break;
        }
    }
```

- If we encounter a digit, we are getting an operand. We compute the numeric value of one or more digits encountered.

```
    else if(ch >= '0' && ch <= '9')
    {
        is_num = 1;
        val = val * 10 + ch - '0';
    }
```

- If the number ends , then numeric value is pushed into the stack

```
    else if (ch == ' ' && is_num)
    {
        printf("pushing %d\n", val);
        push(&mystack, val);
        val = 0;
        is_num = 0;
    }
}
```

- When the input string has been completely processed, there should be one element on the stack – which is the result.

```
printf("result : %d\n", pop(&mystack));
```

Algorithm for conversion of infix to postfix:

We have discussed the advantage of having an expression in postfix. The evaluation becomes simple. There is no hierarchy. This also implies that there are no parentheses.

How do we convert an expression in infix to an expression in postfix? To do this, we use a stack of operators.

Let us discuss a few simple examples before we go through the algorithm in detail.

Example 1 :

3 + 4

Whenever we get operands, we just output them to the postfix expression.

When we encounter an operator, if the stack is empty push the operator on to the stack.

When all the operands are processed, pop from the stack into the postfix expression until the stack becomes empty.

Output 3; push + ; output 4; pop + :

Postfix expression : 3 4 +

Example 2 :

4 * 5 + 6

As per the earlier example, we do the following steps.

Output 4; push *; output 5

We encounter + in the string and we have * on the stack. Since * has a higher precedence compared to +,

pop * and push +.

Output 6

Pop the stack until empty

Pop +

Postfix expression : 4 5 * 6 +

Example 3 :

4 + 5 * 6

As per the earlier example, we do the following steps.

Output 4; push +; output 5

We encounter * in the string and we have + on the stack. Since * has a higher precedence compared to +, we push * onto the stack.

Push *

Output 6

Pop all operators from the stack

Output *

Output +

Postfix expression : 4 5 6 * +

Example 4 :

5 * 6 + 7 * 8

As per the earlier example, we do the following steps.

Output 5; push *; output 6

We encounter + in the string and we have * on the stack. Since * has a higher precedence compared to +, pop * and push +

Pop *; push +

Output 7

We encounter * in the string and we have + on the stack. Since * has a higher precedence compared to +, we push * onto the stack.

Output 8

Pop all operators

Pop *

Pop +

Postfix expression : 5 6 * 7 8 * +

Example 5 :

What if the infix expression has parentheses? There are no parentheses in postfix expression as there is no precedence.

We treat the parentheses differently. We want to always push the left parenthesis onto the stack. We will give left parenthesis a very high precedence value in the string so that it always gets pushed in. Once it is in the stack, we want all other operators to get pushed in until the right parenthesis is encountered. So, we associate a very low precedence value to the left parenthesis once it is in the stack.

When we encounter a right parenthesis, we pop all operators until we encounter left parenthesis on the stack to the output. We then pop the left parenthesis out.

3 * (4 + 5) * 6

Output 3

Push *

(in the string has very high precedence

Push (

Output 4

(in the stack has very low precedence

Push +

Output 5

Encountering } in the string

Pop until left parenthesis

Pop +

Equal precedence

If left associative, pop from the stack and push the operator in the string

Else push the operator from the string

Pop * from the stack

Push * from the string

Output 6

Pop * from the stack

Output expression : 3 4 5 + * 6 *

Algorithm:

Let us examine the algorithm:

- Create input string; read it
- Create an output string
- Provide mechanisms to walk through both

```
char in[MAXSIZE + 1]; char *s = in;
char post[MAXSIZE + 1]; char *p = post;
gets(in);
printf("input expression : :%s:\n", s);
```

- Create a stack of operators. Initialize

```
stack_t mystack;
init_stack(&mystack);
```

- Walk through the string

```
char ch;
char ch1;
while( (ch = *s++) != '\0')
{
```

  - On right parenthesis, pop all operators to output until the left parenthesis is encountered on the stack. Remove that as well.

```
if(ch == ')')
{
        while( (ch1 = pop(&mystack) ) != '('
        {
                *p++ = ' ';
                *p++ = ch1;
        }
}
```

  - On an operator, if the stack is empty, push it

```
else if(ch == '(' || ch == '+' || ch == '-' ||
        ch == '*' || ch == '/')
```

```
        {
                if(is_empty(&mystack))
                {
                        push(&mystack, ch);
                }
```

- Otherwise compare the precedence of operator in string and

  The precedence of operator in the stack

  Decide to push or pop.

  Why a loop ?

```
                else
                {
                        while(!is_empty(&mystack) &&
                                prec_in_str(ch) < prec_in_stack(peek(&mystack)))
                        {
                                *p++ = ' ';
                                *p++ = pop(&mystack);

                        }
                        push(&mystack, ch);
                }
        }
```

- Output operands directly

```
        else
        {
                *p++ = ch;
        }
}
```

Pop operators from the stack to the output

```
while(! is_empty(&mystack))
{
        *p++ = ' ';
        *p++ = pop(&mystack);
}
```

```
*p = '\0';
```

- Display the string

```
printf("infix : %s\n", in);
printf("postfix : %s\n", post);
```

So, we have learnt how to play with data structure as if it is another type.

Is it not?

Multilist and Skiplist

Concept of ADT : Revision :

So far, we have learnt two data structures list and stack. We have realized that these data structures can be implemented in different ways – but they support the interface. Basically, we have created a new type. Once we create them, we can use them like any other built-in type. So, we have enhanced the language.

We have realized that separating the concerns leads to a good software development. A data structure has number of components. A data structure exhibits certain behaviour independent of the component. Each component has its own behaviour which does not depend on the data structure where it is included.

A polynomial has number of terms. A term has coefficient and exponent. We could represent the polynomial using an array of terms or list of terms. Term should be independent of the data structure which contains it. Therefore making the term self-referential is wrong. Think of an array of terms. If it has a pointer to itself, what can we put there?

Similarly if we implement the polynomial using a list, the list should know nothing of the term. The list should provide way to initialize, traverse, insert and so on. Whenever any behaviour of the component is required, it should invoke the suitable function of the component.

Trust these points are clear. We will try to emphasize this throughout the course.

Skiplist :

Let us consider that I want to travel to a village between Srirangapatna and Mysore. We may have shuttle buses from Bangalore to that village. It will stop at every village. It will take lot of time to reach my destination.

Can we take an express bus to Mysore and then come back?

Can we take a direct bus to Srirangapatna which does not stop at every village and then take a local bus to the village from Srirangapatna?

Yes. Definitely. This will save time.

This concept is called a skip list.

Limitation of a normal list:

Let us consider an example of keys arranged in non decreasing order.

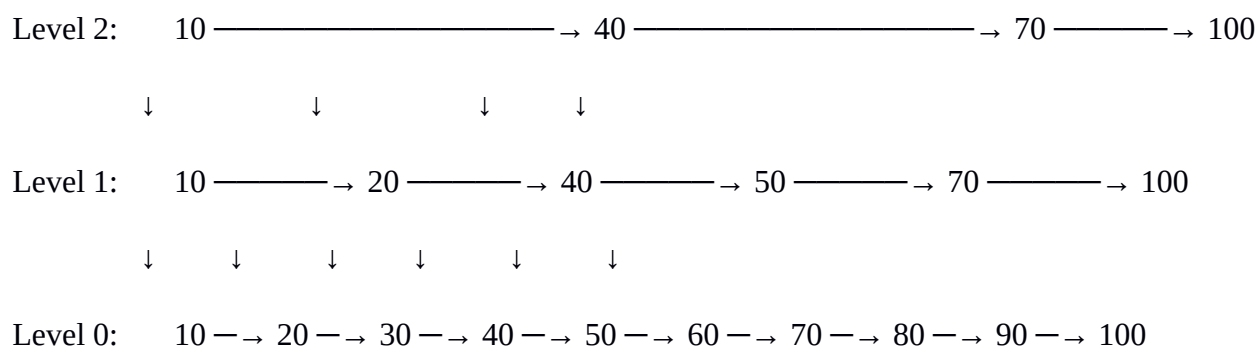
10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100 → NULL

Searching for 90 would require that we start from the first node and traverse almost the whole length of the list.

In the worst case, the number of steps and comparisons will be proportional to the number of elements in the list.

Structure of skiplist :

Can we arrange this in multiple levels?



In level2, we start at the node having key 10 and then jump to the node having key 40 and then 70 and then 100. We cover the whole list in a few steps – skip part of the list in each jump.

In level1, we access more nodes while traversing compared to level 2.

In level0, we walk through the list one step at a time.

Searching in a skiplist :

Let us say that I am searching for 90.

We start level 2.

Level 2: 10 ——————→ 40 ——————→ 70 ——————→ 100

↓ ↓

Level 1: 10 ——→ 20 ——→ 40 ——→ 50 ——→ 70 ——→ 100

↓ ↓

Level 0: 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100

Step 1

Start at 10 at the highest level.

10 ——————→ 40 ——————→ 70 ——————→ 100

^

10 < 90, so continue

Step 2

Move to 40.

40 < 90, so continue.

Step 3

Move to 70.

70 < 90, so continue.

Step 4

The next node is 100.

But:

100 > 90

So we will not find 90 here..

We drop down a level at 70.

Level 2: 10 ————→ 40 ————→ 70 ————→ 100

↓

Level 1: 10 —→ 20 —→ 40 —→ 50 —→ 70 —→ 100

Level 0: 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100

Now at Level 0:

70 → 80 → 90

We reach 90.

So instead of examining all nine elements, the higher levels allowed us to **jump over large portions of the list**.

Structure of nodes and other operations on the skiplist:

A skip list may have any number of levels. The number of levels depends on the expected number of nodes in the list. Normally, each higher level will have half the nodes of the next level.

Each node will have pointers for each level. Only those pointers which participate in the list at that level shall be populated.

Insertion, deletion will again follow the concept we discussed in searching. We always start from top and locate all the levels where the new node has to be inserted or deleted.

Note: Implementing a skiplist is not part of the syllabus.

Applications:

- In Memory ordered data for faster insertion, deletion and searching
- Concurrent data structures where operations can happen in parallel
- Priority ordered events

  The skiplists can be used instead of balanced search trees.

Salient points:

A skip list is a sorted linked list augmented with multiple levels of forward pointers. Higher levels allow searches to skip over many elements**.**

1. Bottom level contains all elements.
2. Higher levels contain fewer elements.
3. Higher levels provide shortcuts.

MultiLists:

This means a list of lists. It is possible that a node may be part of more than one list. It is also likely that a list has nodes whose keys are heads of some other list.

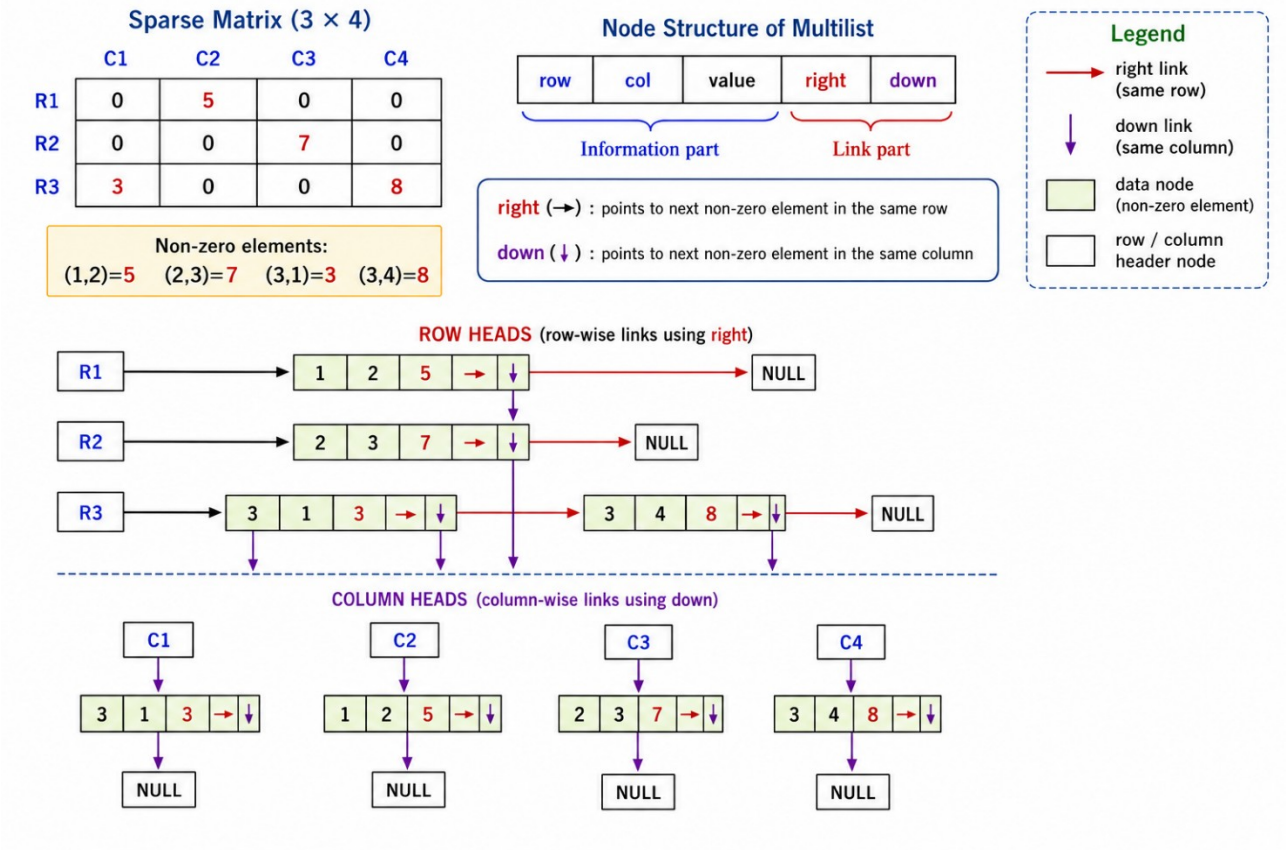
Sparse matrix : time space trade off.

Let us say that we have a huge matrix – 10000 X 10000 elements. These are not uncommon in images. This also happens in techniques called finite element methods or finite differences required in Engineering Applications. This also manifests in Electrical Engineering having huge networks.

These matrices would require a lot of memory. But most of the elements are zero. Can we store only non zero elements? In that case, how to identify the row and the column to which the element belongs to? Can we store the value, the row number and the column number? It is fine, how to traverse rowwise and/or columnwise? Certain operations like addition and multiplication of matrices would require such traversals.

How about making a rowlist for each row which connects every element in that row and a column list for each column which connects every element in that column? Do you observe that in such a case, each element would occur in two lists?

Such a list is called a multilist. It is a list of lists. One possibility is that in such a case, the element occurs in more than one list.



The diagram above depicts how a sparse matrix would store only nonzero values. For clarity, the same node is shown separately in a row list as well as a column list. In practice, there will be only one node for each non zero value.

Row heads and column heads may themselves be stored in a list or an array.

The same node can be reached through different linked lists, each representing a different relationship**.**

This example illustrates how we can gain in memory at the cost of time. Another similar example is file compression.

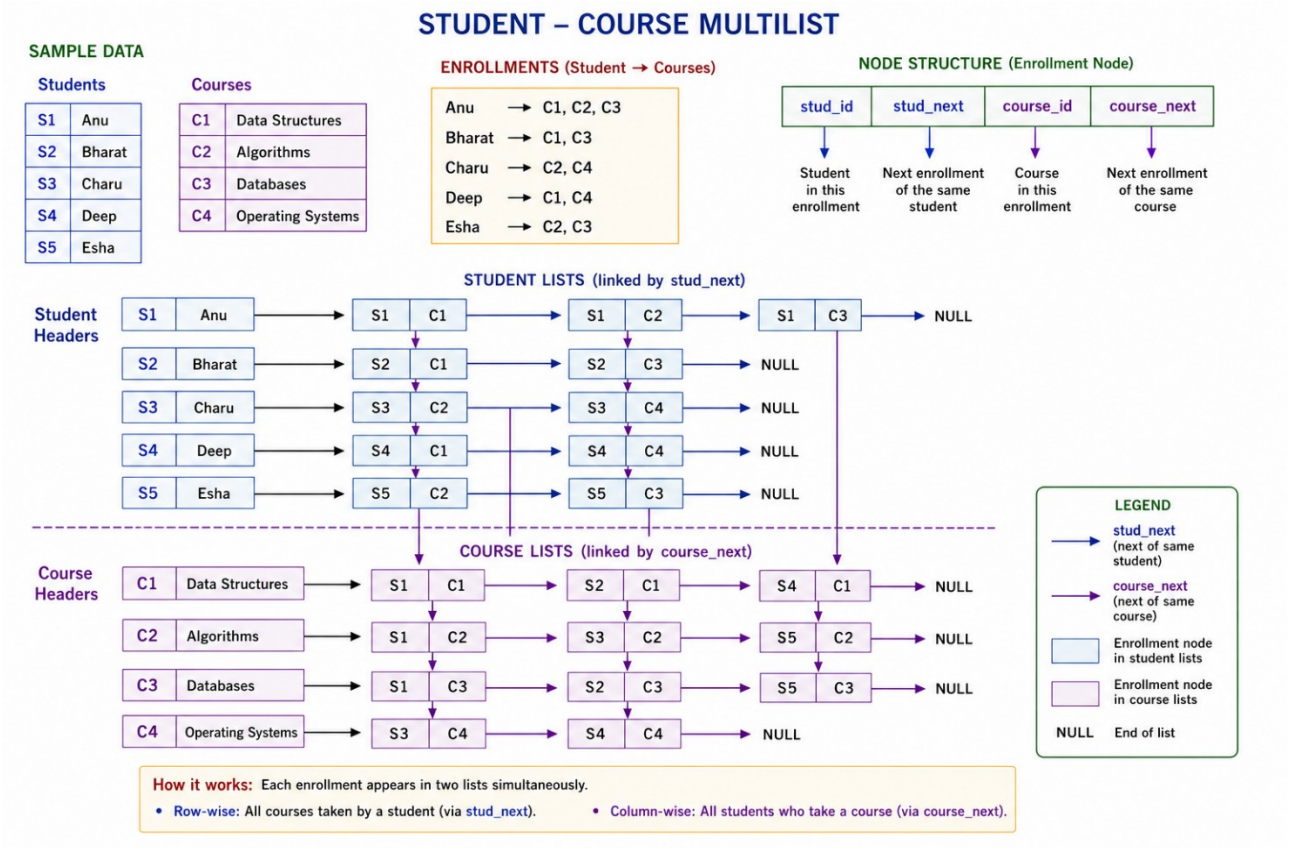
One physical node, multiple logical views:

Let us take an example of students taking a few courses and each course having a few students. It may be necessary to find the courses each student has taken or find the students who have taken a particular course.

One possible solution would be to keep two data structures. Map each student to the list of courses taken by him. Map each course to a list of students. What are the issues in this approach?

This would require more memory. Any change would require updation at multiple places. If the data is not updated at each occurrence, integrity is lost.

The solution would be store the data of student and his course at only one place and allow for traversal student wise or subject wise. This requires a multilist.



In this diagram, the nodes are shown separately. But there will be only one node for each student, subject mapping.

Applications of multilists:

1. Sparse matrix
2. Graph representation
3. Social networks
4. Multiple views as in files.

Salient points :

- Multiple relations
- Avoids duplicates
- Avoids multiple updation
- Complicated pointer management
- Complicated insertion/deletion